



UNIVERSITÉ ABDELMALEK ESSAADI
FACULTÉ DES SCIENCES ET TECHNIQUES
DE TANGER
DÉPARTEMENT D'INFORMATIQUE
Année universitaire 2025-2026



MASTER SCIENCES ET TECHNIQUES
INTELLIGENCE ARTIFICIELLE ET SCIENCES DES
DONNÉES

RAPPORT TECHNIQUE DE PROJET

STAGE PFA : Smart Water Quality System

**Application web full-stack pour laboratoire de traitement
des eaux**

Réalisé par :

- ACHRAF EL BOUMASHOULI
- GHIZLANE AFAILAL TREBAK

Encadré par :

- Pr. Dr IKRAM Ben Abdelouahab
- Pr. Anasse Chafik
- Pr. Amal Maurady

TABLE DES MATIÈRES

CHAPITRE I — Conception et réalisation du système « Smart Water Quality System »

1. Vue d'ensemble du projet

- 1.1 Objectifs du projet
- 1.2 Pile technologique complète

2. Conception UML

- 2.1 Diagramme de cas d'utilisation
- 2.2 Diagramme de classes
- 2.3 Diagramme de séquence — Gestion des Studies & Cycles
- 2.4 Diagramme de séquence — Monitoring & Alerts

3. Architecture fonctionnelle et technique

- 3.1 Architecture technique
- 3.2 Architecture frontend
- 3.3 Architecture backend

4. Architecture de la base de données

- 4.1 Schéma relationnel (résumé)
- 4.2 Tables principales

5. Fonctionnalités détaillées

- 5.1 Gestion des études (Studies)
- 5.2 Gestion des cycles de laboratoire
- 5.3 Résultats de cycle et calculs statistiques
- 5.4 Import de cycles depuis Excel
- 5.5 Collecte automatique des données de capteurs (ThingSpeak)
- 5.6 Détection et gestion des alertes
- 5.7 Notifications par e-mail et Telegram
- 5.8 API REST (documentation interactive)

6. Configuration de l'environnement avec le fichier .env

- 6.1 Configuration du backend
- 6.2 Configuration générale de l'application
- 6.3 Configuration des capteurs ThingSpeak
- 6.4 Configuration de la collecte automatique
- 6.5 Configuration du fuseau horaire
- 6.6 Configuration du système d'alertes par e-mail
- 6.7 Configuration des notifications Telegram
- 6.8 Seuil de détection des alertes
- 6.9 Configuration du frontend
- 6.10 Avantages de cette approche

7. Flux de travail principaux

- 7.1 Création d'une étude et de son premier cycle
- 7.2 Import de cycles depuis Excel
- 7.3 Cycle de vie d'une alerte

8. Guide technique et théorique — pour compléter ou modifier le projet

- 8.1 Théorie et implémentation des calculs statistiques
- 8.2 Modèle de données détaillé
- 8.3 Théorie et implémentation de la gestion des fuseaux horaires
- 8.4 Théorie de la machine à états des alertes
- 8.5 Théorie de l'idempotence dans la collecte de capteurs
- 8.6 Vocabulaire canonique et validation Pydantic
- 8.7 Duplication contrôlée de la logique métier

9. Guide pratique : comment modifier ou étendre le projet

- 9.1 Ajouter un neuvième paramètre de qualité de l'eau
- 9.2 Ajouter un set de capteurs ThingSpeak supplémentaire
- 9.3 Implémenter la détection d'anomalies
- 9.4 Ajouter les pages frontend manquantes
- 9.5 Modifier le comportement d'import Excel

10. Référence API complète (paramètres, statuts, exemples)

11. Variables d'environnement et configuration — référence complète

12. Glossaire des termes techniques et métier

CHAPITRE II — Conception et réalisation du système d'authentification et de gestion des utilisateurs

1. Introduction

2. Présentation générale du module

3. Analyse des besoins

- 3.1 Acteurs du système
- 3.2 Besoins fonctionnels
- 3.3 Besoins non fonctionnels

4. Conception de l'architecture

- 4.1 Style architectural
- 4.2 Organisation des dossiers du projet

5. Implémentation du Backend

- 5.1 Couche routes
- 5.2 Couche contrôleurs
- 5.3 Couche services
- 5.4 Couche modèle de données

6. Conception et gestion de la base de données

6.1 Structure de la table users

7. Développement du Frontend

7.1 Gestion de l'état d'authentification

7.2 Routage protégé

7.3 Composants et pages principaux

8. Fonctionnalités de sécurité

8.1 Hachage des mots de passe

8.2 Authentification par JWT et cookies httpOnly

8.3 Contrôle d'accès basé sur les rôles (RBAC)

8.4 Jetons à usage unique pour les opérations sensibles

8.5 Limitation de débit (rate limiting)

8.6 Prévention de l'énumération des comptes

8.7 Séparation des privilèges pour la modification des comptes

8.8 Protection contre l'injection SQL et l'injection HTML

8.9 Configuration par variables d'environnement

9. Processus d'inscription et de connexion

9.1 Inscription (POST /api/auth/register)

9.2 Connexion (POST /api/auth/login)

9.3 Déconnexion (POST /api/auth/logout)

10. Gestion des comptes administrateurs

10.1 Depuis le tableau de bord administrateur

10.2 Depuis un e-mail : approbation/rejet en un clic

11. Gestion de la récupération du mot de passe

12. Communication par email

13. API et flux de communication

14. Tests et validation du système

15. Déploiement et configuration

16. Conclusion

Chapitre I — Conception et réalisation du système

« Smart Water Quality System »

1. Vue d'ensemble du projet

Le système « Smart Water Quality System » est une application full-stack destinée à un laboratoire de suivi du traitement des eaux usées. Il assure en continu la collecte de mesures de capteurs via ThingSpeak, organise ces données au sein d'« études » expérimentales, permet aux opérateurs de laboratoire de saisir et de calculer des « cycles » de traitement manuels (série d'eau brute, série plantée, série témoin) sur 8 paramètres de qualité de l'eau, prend en charge l'import massif de cycles depuis Excel, et déclenche des alertes lorsque la collecte de données s'interrompt ou que les capteurs renvoient des valeurs manifestement invalides.

Le projet est composé de deux applications indépendantes qui communiquent exclusivement par une API REST/JSON :

- Backend — un service FastAPI qui interroge ThingSpeak selon une planification pour ingérer les données brutes des capteurs (pH, température, conductivité électrique EC, turbidité, oxygène dissous DO) dans PostgreSQL, expose une API REST pour gérer les études et les cycles de laboratoire, calcule les statistiques des cycles (moyenne, écart-type, pourcentage d'abattement), prend en charge l'import de cycles en masse depuis Excel, et détecte/enregistre des alertes lorsque la collecte échoue ou que les capteurs renvoient des lectures dégénérées.
- Frontend — un tableau de bord Next.js qui permet à un opérateur de créer des études, de saisir ou d'importer des données de cycle, de consulter les résultats calculés, et de surveiller/configurer les alertes, le tout via l'API REST du backend.

1.1 Objectifs du projet

Tels qu'ils ressortent concrètement de l'implémentation :

- Ingérer de façon continue et fiable les mesures de capteurs depuis N canaux ThingSpeak configurés (« sets de capteurs »), sans effort manuel.
- Offrir aux opérateurs une manière structurée de saisir les résultats d'un cycle de traitement de 24 lignes (8 paramètres × 3 étapes) par étude expérimentale, à la main ou via un unique import Excel, avec la même logique de calcul appliquée dans les deux cas.
- Calculer automatiquement les statistiques que les opérateurs devraient sinon calculer dans un tableur : moyenne, écart-type d'échantillon, et pourcentage d'abattement (removal %) par rapport à l'échantillon brut non traité.
- Détecter et signaler, sans surveillance manuelle, deux conditions de défaillance concrètes du pipeline de capteurs : une absence totale de nouvelles données, et des capteurs renvoyant des lectures entièrement nulles (clairement invalides) — avec une sensibilité configurable et une notification optionnelle par e-mail/Telegram.
- Conserver l'identité des plantes utilisées dans une expérience (quelle espèce a été testée) comme une donnée de premier ordre, propre à chaque cycle, indépendante des mesures expérimentales elles-mêmes.

1.2 Pile technologique complète

Couche	Technologie
Framework frontend	Next.js 15.5 (App Router), React 18, TypeScript 5
Style frontend	Tailwind CSS 3
Couche de données frontend	fetch natif via un client typé (lib/api.ts)
Framework backend	FastAPI 0.110, Uvicorn 0.27
ORM backend	SQLAlchemy 2.0
Validation backend	Pydantic 2 / pydantic-settings
Base de données	PostgreSQL (pilote psycopg2-binary)
Migrations	Alembic 1.13
Planification	APScheduler 3.10 (BackgroundScheduler, in-process)
Traitement de données (backend)	pandas + openpyxl (import Excel, export CSV)
HTTP sortant (backend)	requests (ThingSpeak, API Bot Telegram)
E-mail	bibliothèque standard Python smtplib / email.mime.text

2. Conception UML

2.1 Diagramme de cas d'utilisation

Description

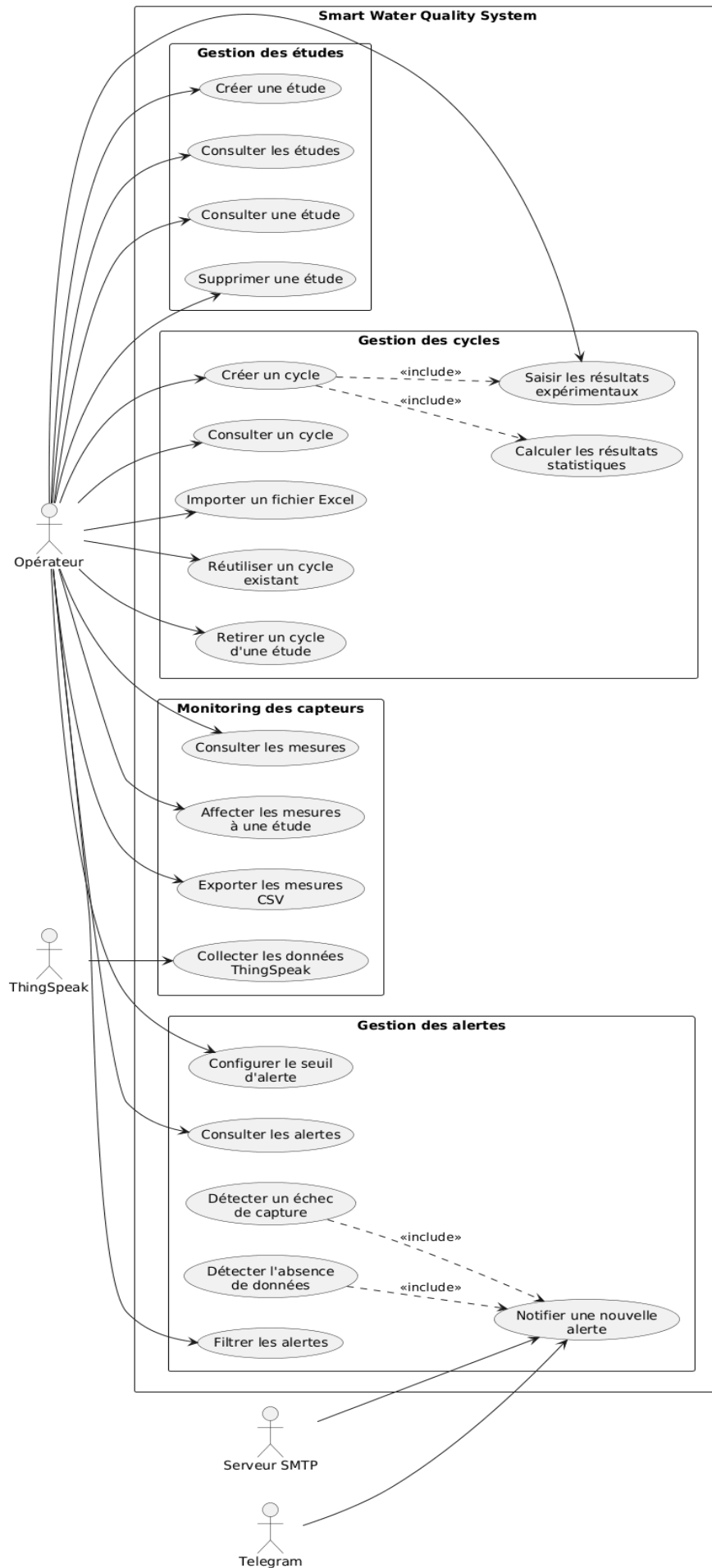
Le diagramme de cas d'utilisation présente les principales fonctionnalités offertes par le système Smart Water Quality System et les interactions entre les différents acteurs et l'application.

Le système est principalement utilisé par un Opérateur, qui peut gérer les études et les cycles expérimentaux, consulter les mesures des capteurs, associer des données à une étude, exporter les mesures et gérer les alertes. La collecte automatique des données depuis ThingSpeak constitue également une fonctionnalité importante du système.

Le diagramme regroupe les fonctionnalités en quatre domaines principaux : gestion des études, gestion des cycles, monitoring des capteurs et gestion des alertes.

Diagramme

Diagramme de cas d'utilisation - Smart Water Quality System



Explication

Le principal acteur du système est l'Opérateur. Il interagit avec l'application afin d'effectuer les opérations nécessaires au suivi des expériences et à la surveillance de la qualité de l'eau.

Gestion des études

La première partie du diagramme concerne la gestion des études. L'opérateur peut :

- Créer une étude, en renseignant notamment son nom ainsi que ses dates de début et de fin ;
- Consulter les études, afin d'accéder à la liste des études existantes ;
- Consulter une étude, afin d'afficher ses informations et les cycles qui lui sont associés ;
- Supprimer une étude, lorsque celle-ci n'est plus nécessaire.

Une étude constitue le point d'entrée principal pour organiser les expérimentations. Elle peut être associée à plusieurs cycles grâce à la relation StudyCycle.

Gestion des cycles

La deuxième partie concerne les cycles expérimentaux. L'opérateur dispose de plusieurs possibilités :

- Créer un cycle manuellement ;
- Consulter un cycle et ses résultats ;
- Importer un fichier Excel contenant plusieurs cycles ;
- Réutiliser un cycle existant dans une autre étude ;
- Retirer un cycle d'une étude.

Lors de la création d'un cycle, l'opérateur saisit les résultats expérimentaux. Le système calcule ensuite automatiquement les statistiques nécessaires, notamment la moyenne, l'écart-type et, lorsque cela est applicable, le pourcentage d'abattement (Removal %). Les cycles sont constitués de 24 lignes correspondant aux combinaisons de paramètres et d'étapes expérimentales.

Le diagramme représente également les relations <<include>> entre Créer un cycle, Saisir les résultats expérimentaux et Calculer les résultats statistiques. Cela signifie que la saisie des résultats et le calcul des statistiques font partie du processus de création d'un cycle.

La fonctionnalité Réutiliser un cycle existant permet d'associer un cycle déjà présent dans la base à une nouvelle étude. Le cycle n'est pas dupliqué : seule l'association entre l'étude et le cycle est créée.

De la même manière, Retirer un cycle d'une étude ne supprime pas le cycle de la base. Seule son association avec l'étude concernée est supprimée. Les résultats et les plantes associés au cycle sont conservés.

Monitoring des capteurs

La troisième partie concerne la surveillance des données issues des capteurs. L'opérateur peut :

- Consulter les mesures enregistrées ;
- Affecter les mesures à une étude, afin de rattacher les données de capteurs à une étude donnée ;
- Exporter les mesures CSV pour une analyse externe.

La collecte des données ThingSpeak est réalisée automatiquement par le système.

Gestion des alertes

La dernière partie concerne la supervision des problèmes liés à la collecte des capteurs. Le système permet de :

- Configurer le seuil d'alerte ;
- Consulter les alertes ;
- Filtrer les alertes ;
- Détecter un échec de capture ;
- Détecter l'absence de données ;
- Notifier une nouvelle alerte.

Les deux détecteurs principaux surveillent respectivement l'absence de nouvelles données et les captures dont les cinq paramètres de mesure sont simultanément égaux à zéro. Lorsque le nombre d'échecs consécutifs atteint le seuil configuré, une alerte peut devenir active.

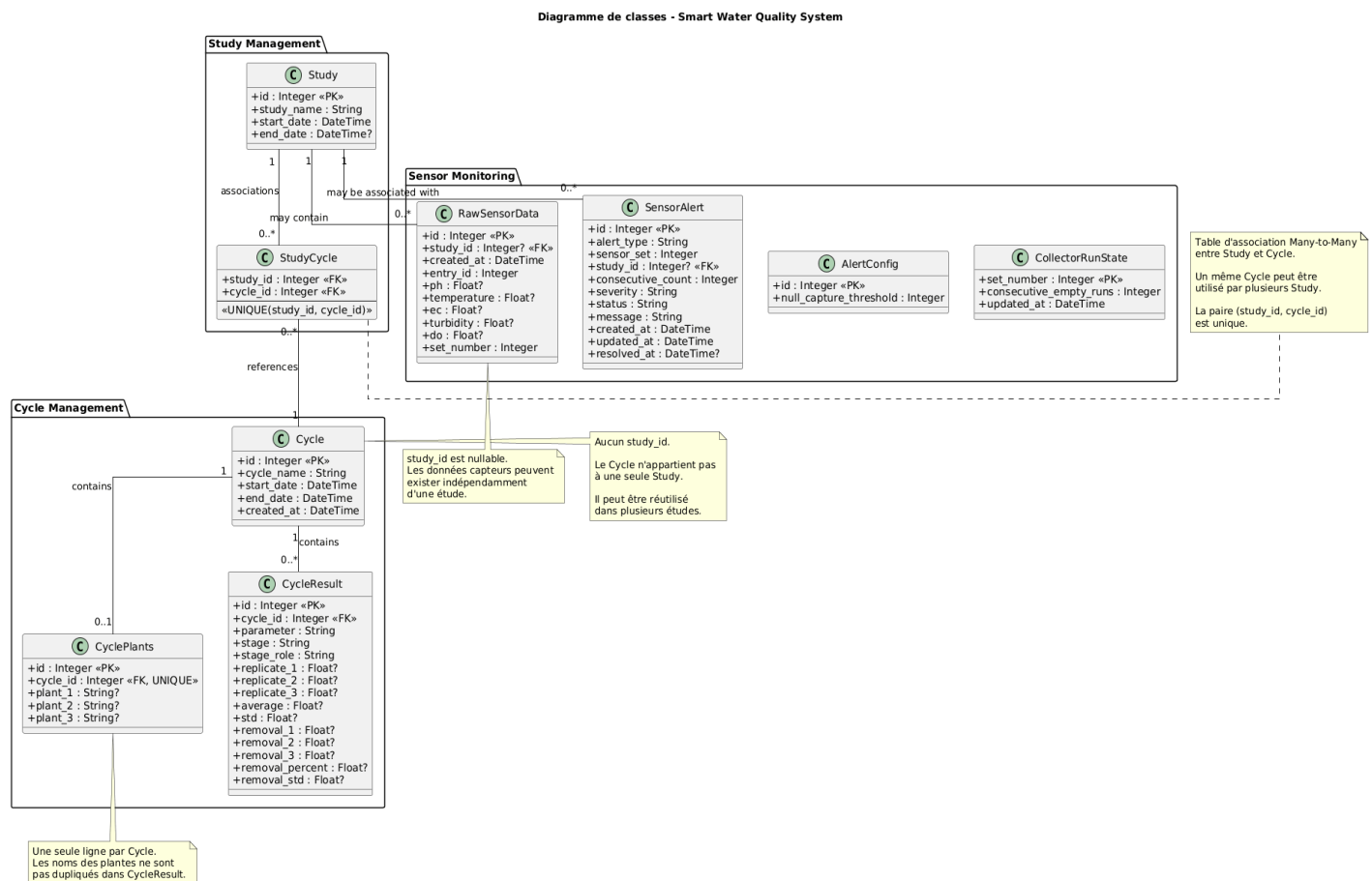
Les notifications peuvent être envoyées par e-mail ou via Telegram, selon la configuration disponible.

2.2 Diagramme de classes

Description

Le diagramme de classes représente la structure statique du système, notamment les principales entités métier, leurs attributs, leurs clés étrangères ainsi que les relations entre elles. Il correspond au modèle de données implémenté avec SQLAlchemy et PostgreSQL.

Diagramme



Explication

Le modèle de données est organisé autour de deux domaines principaux : la gestion des études et des cycles et le monitoring des capteurs.

Classe Study

Study représente une étude expérimentale. Elle contient son identifiant, son nom ainsi que ses dates de début et de fin.

Une étude peut être associée à plusieurs cycles et à plusieurs données brutes de capteurs.

Classe StudyCycle

StudyCycle est la table d'association entre Study et Cycle. Elle contient study_id et cycle_id, avec une contrainte d'unicité sur leur combinaison.

Elle permet d'établir une relation many-to-many : une étude peut utiliser plusieurs cycles et un même cycle peut être réutilisé dans plusieurs études. Le Cycle reste ainsi indépendant de l'étude.

Classe Cycle

Cycle représente un cycle expérimental avec son identifiant, son nom, ses dates et sa date de création.

Il est indépendant de Study et peut être associé à plusieurs études grâce à StudyCycle, ce qui permet sa réutilisation sans duplication.

Classe CycleResult

CycleResult contient les résultats expérimentaux d'un cycle : paramètre, étape, réplicats, moyenne, écart-type et indicateurs de removal.

Un cycle peut contenir plusieurs résultats. Un cycle complet comporte 24 lignes, correspondant à 8 paramètres × 3 étapes. Les statistiques sont calculées automatiquement avant l'enregistrement.

Classe CyclePlants

CyclePlants contient les plantes utilisées dans un cycle (plant_1, plant_2, plant_3).

Chaque cycle peut avoir au maximum un enregistrement CyclePlants. Les informations sur les plantes sont ainsi séparées des résultats expérimentaux.

Classe RawSensorData

RawSensorData représente les mesures brutes des capteurs : pH, température, conductivité, turbidité et oxygène dissous, ainsi que les informations temporelles et le numéro du set.

`study_id` étant nullable, les mesures peuvent être collectées indépendamment d'une étude puis associées ultérieurement. Une contrainte sur (`entry_id`, `set_number`) empêche les doublons.

Classe SensorAlert

SensorAlert représente les alertes générées par le système de monitoring. Elle conserve notamment le type d'alerte, le set de capteurs, la sévérité, le statut et les informations temporelles.

Les principaux types sont `no_data` et `capture_failure`, avec les états `active` et `resolved`.

Classe AlertConfig

AlertConfig contient la configuration du système d'alertes, notamment `null_capture_threshold`, qui définit le nombre d'échecs consécutifs nécessaire avant l'activation d'une alerte.

Classe CollectorRunState

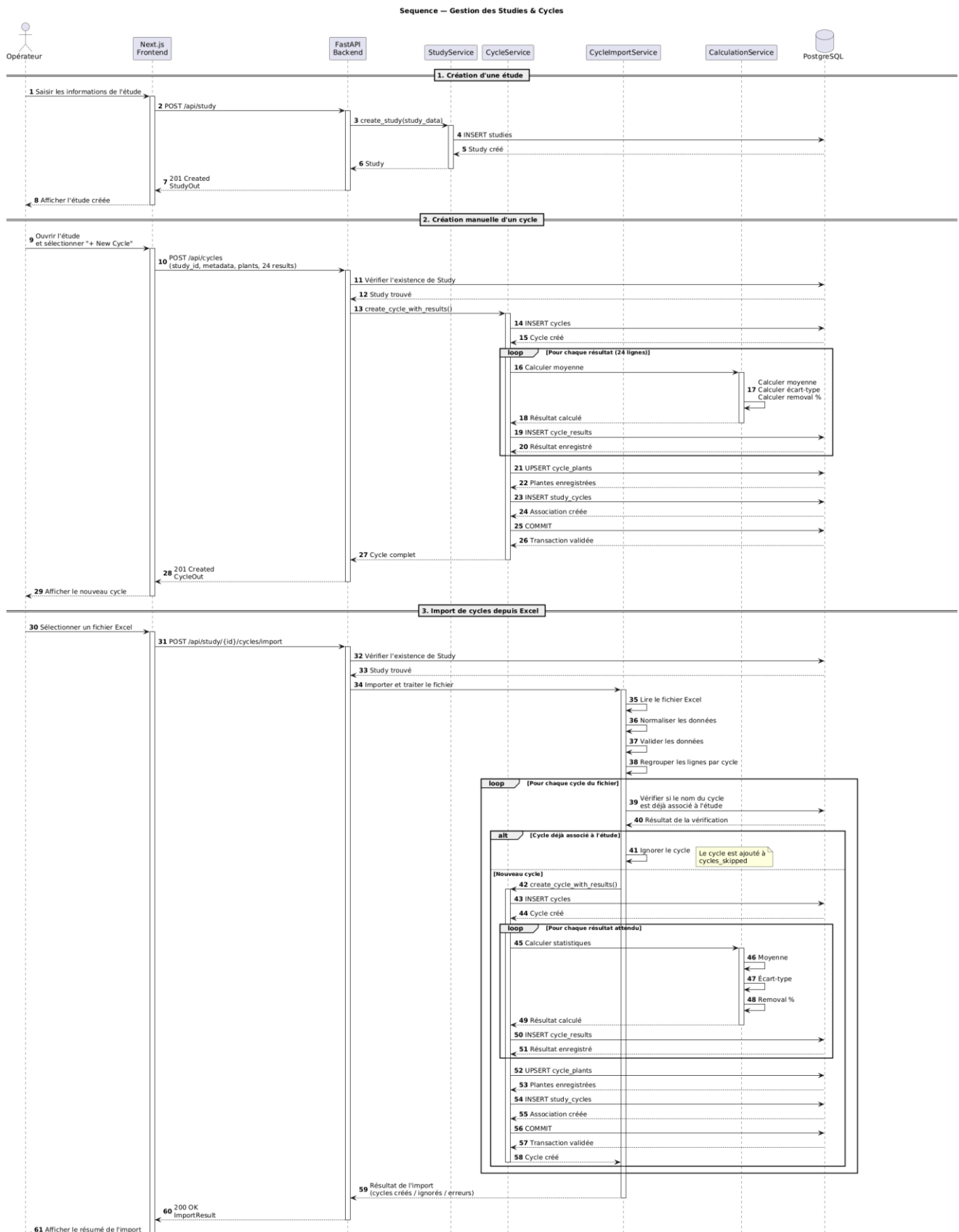
CollectorRunState conserve l'état du collecteur pour chaque set de capteurs. Son attribut `consecutive_empty_runs` permet de suivre les collectes consécutives sans nouvelles données et participe à la détection des alertes.

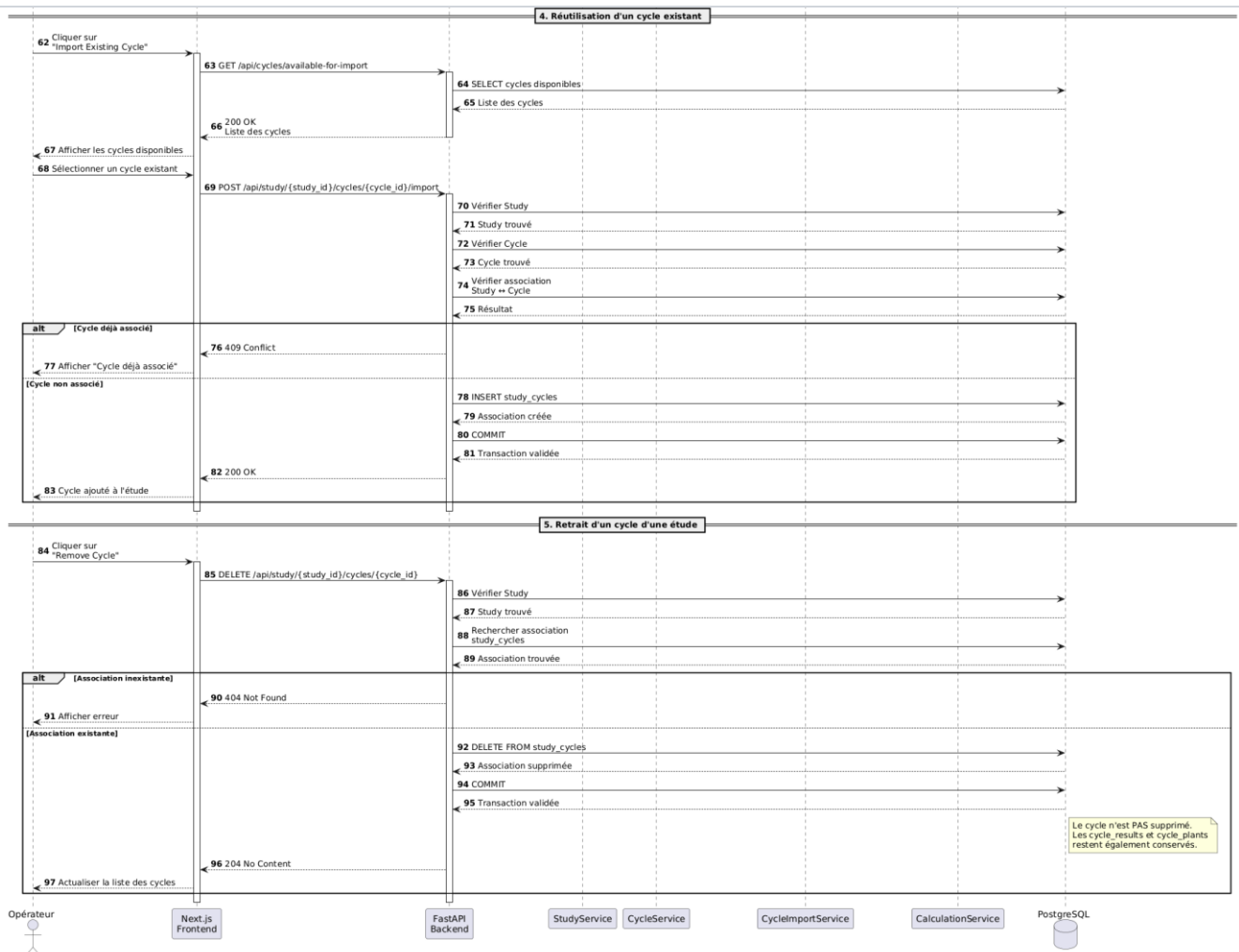
2.3 Diagramme de séquence — Gestion des Studies & Cycles

Description

Ce diagramme de séquence présente les principaux scénarios liés à la gestion des études et des cycles expérimentaux. Le diagramme montre les interactions successives entre l'opérateur, le frontend Next.js, le backend FastAPI, les services métier, le service de calcul et la base de données PostgreSQL.

Diagramme





Explication

Lors de la création d'une étude, le frontend Next.js transmet les informations au backend FastAPI qui les enregistre dans PostgreSQL.

Pour la création d'un cycle, le backend vérifie l'étude, crée le cycle et traite ses 24 résultats expérimentaux. Le CalculationService calcule notamment la moyenne, l'écart-type et le pourcentage d'abattement. Les résultats et les plantes sont ensuite enregistrés, puis une association study_cycles est créée dans la même transaction.

L'import Excel suit une logique similaire, avec une validation et un traitement de chaque cycle. Un cycle existant peut également être réutilisé dans une autre étude : dans ce cas, seules les données de la table study_cycles sont modifiées, sans duplication du cycle.

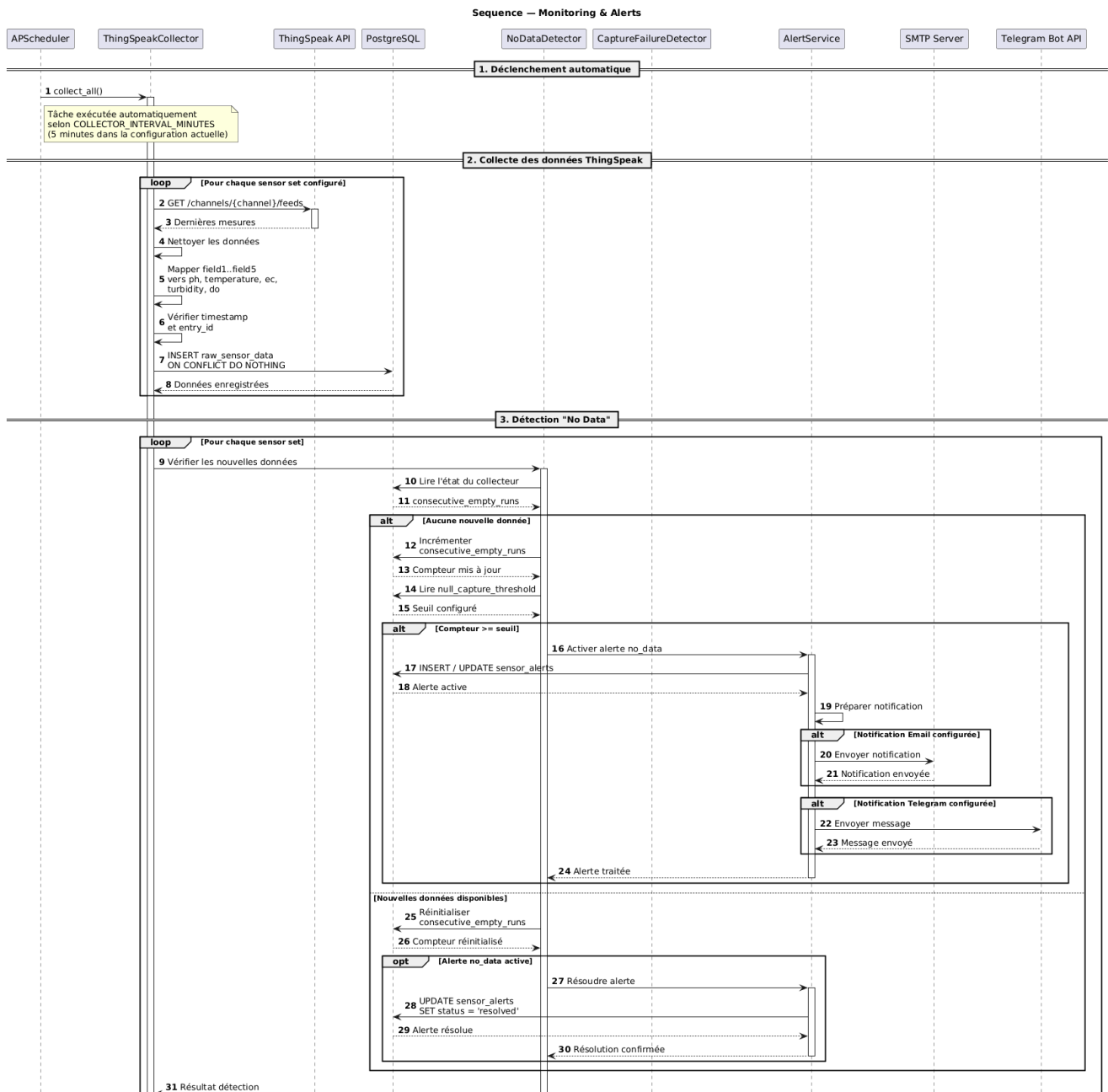
Enfin, le retrait d'un cycle supprime uniquement son association avec l'étude ; le cycle, ses résultats et ses plantes restent conservés.

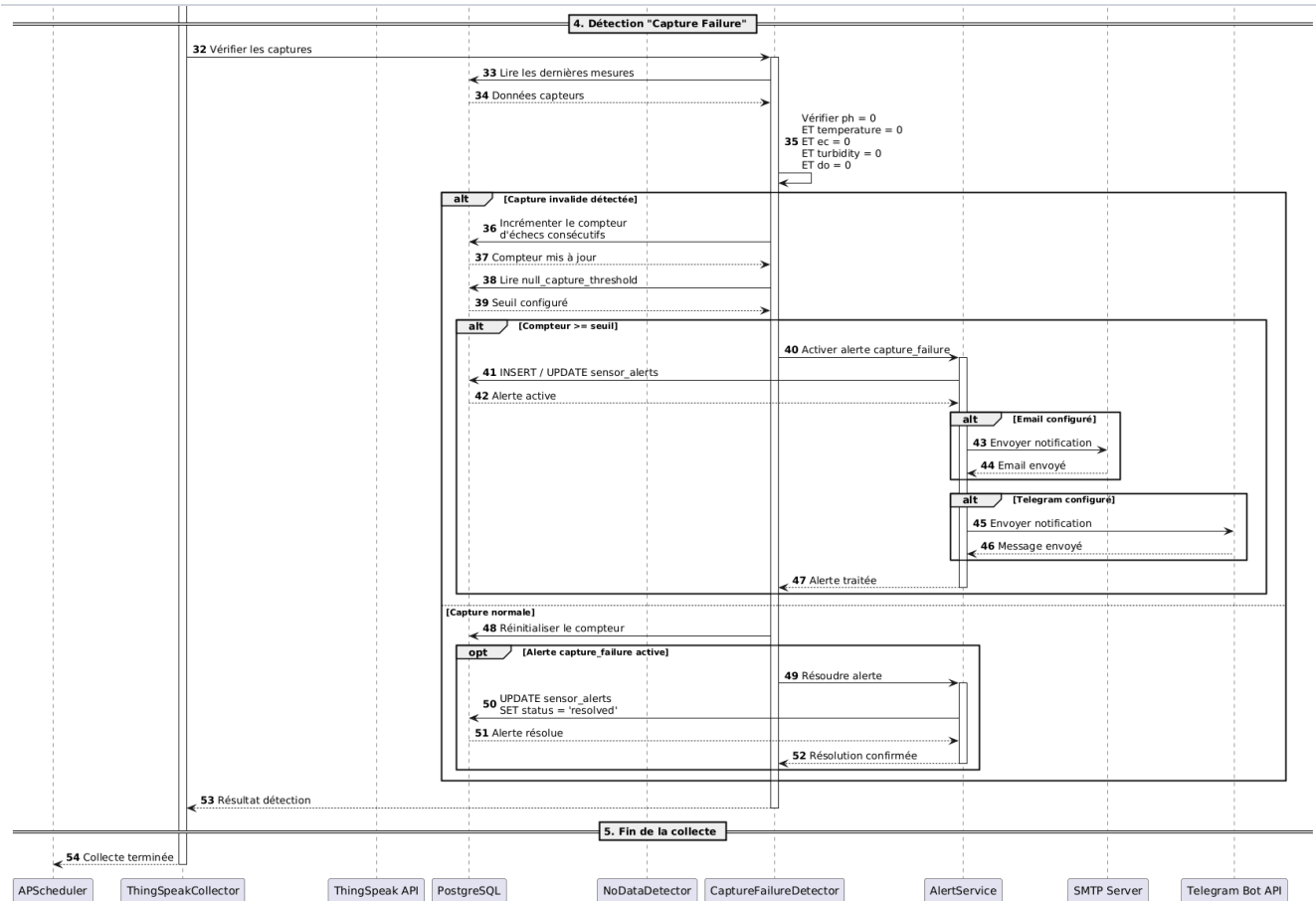
2.4 Diagramme de séquence — Monitoring & Alerts

Description

Ce diagramme décrit le fonctionnement automatique du système de monitoring, depuis la collecte des données ThingSpeak jusqu'à leur stockage et à la génération éventuelle de notifications d'alerte.

Diagramme





Explication

La collecte est déclenchée automatiquement par APScheduler selon l'intervalle configuré. Le ThingSpeakCollector récupère les mesures depuis les différents channels ThingSpeak, nettoie les données puis les enregistre dans raw_sensor_data.

Après chaque collecte, deux situations sont contrôlées : l'absence de nouvelles données et l'échec de capture, lorsque les cinq paramètres mesurés sont simultanément égaux à zéro.

Les compteurs d'échecs sont comparés au seuil configuré. Lorsque celui-ci est atteint, une alerte est créée ou activée dans sensor_alerts. Une notification peut alors être envoyée par SMTP ou Telegram. Lorsque les données redeviennent normales, l'alerte active est automatiquement résolue.

3. Architecture fonctionnelle et technique

Deux flux fonctionnels s'exécutent largement de manière indépendante :

- Flux automatique des capteurs : ThingSpeak → collecteur → table raw_sensor_data → détecteurs d'alertes → (facultativement) un opérateur associe les mesures à une étude, ou les exporte en CSV.
- Flux manuel / cycle de laboratoire : un opérateur (via le formulaire ou un fichier Excel) soumet 24 lignes de mesures en réplicats pour un cycle → le backend calcule les statistiques → les résultats sont persistés et affichés.

3.1 Architecture technique

Le frontend (Next.js 15 / React 18 / TypeScript, Tailwind CSS, <http://localhost:3000>) communique en HTTP/JSON avec le backend (FastAPI / SQLAlchemy 2.0, APScheduler, pandas, <http://localhost:8000>), CORS activé. Le backend communique en SQL (psycopg2) avec PostgreSQL, dont le schéma est géré par Alembic. Le backend effectue également des appels HTTP sortants vers l'API ThingSpeak (collecte des données de capteurs), un serveur SMTP (e-mails d'alerte optionnels) et l'API Bot Telegram (notifications d'alerte optionnelles).

Il n'existe ni file de messages, ni couche de cache, ni microservice séparé : le backend est un unique processus FastAPI qui sert à la fois l'API et exécute la tâche de collecte en arrière-plan dans le même processus, via APScheduler.

3.2 Architecture frontend

Le frontend est une application Next.js App Router où chaque page est un composant client ("use client") ; il n'y a ni récupération de données côté serveur, ni server actions. Les pages appellent directement le backend depuis le navigateur via le client typé de lib/api.ts. L'état est géré localement par useState/useEffect page par page — il n'existe pas de librairie d'état global.

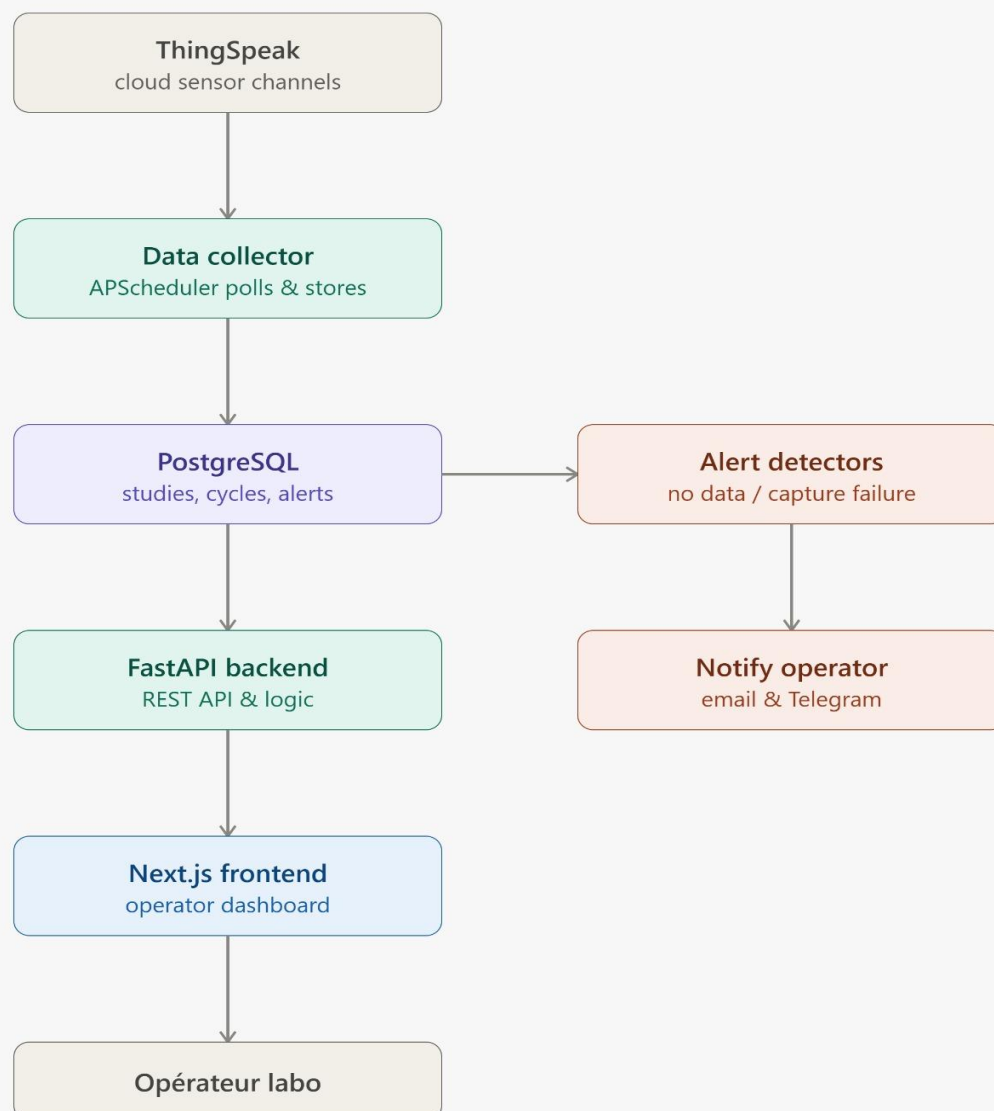
- app/ — les routes, un dossier par segment d'URL, selon les conventions de fichiers Next.js (page.tsx, segments dynamiques [studyId], [cycleId]).
- components/ — cinq composants présentationnels/interactifs partagés entre les pages, centrés sur le tableau de résultats de cycle et les champs/récapitulatifs de noms de plantes.
- lib/ — le client API, les types TypeScript partagés, les constantes du domaine (paramètres/étapes, miroir du backend), les fonctions de calcul utilisées uniquement pour l'aperçu côté client, et le formatage de dates sensible au fuseau horaire.

3.3 Architecture backend

Le backend suit une structure en couches routes → services → modèles, avec des schémas Pydantic validant tout ce qui traverse la frontière HTTP.

- app/main.py assemble l'application FastAPI, le CORS, le gestionnaire d'exceptions global, les quatre modules de routes, et le cycle de vie d'APScheduler (démarrage : enregistrement et lancement de la tâche, avec une collecte immédiate ; arrêt : arrêt du planificateur).

- `app/collectors/collector.py` implémente `ThingSpeakCollector` (récupération → nettoyage → stockage) et `collect_all()`, le point d'entrée unique utilisé à la fois par la tâche planifiée et par l'endpoint manuel `POST /api/collect`.
- `app/services/*` contiennent toute la logique métier : CRUD et assignation des études, export CSV, calculs et persistance des cycles, analyse de l'import Excel, machines à états de détection d'alertes, et envoi des notifications.
- `app/models/*` sont les classes ORM SQLAlchemy correspondant aux 9 tables de la base de données, dont le schéma est créé et géré par une migration Alembic consolidée.
- `app/schemas/*` définissent le vocabulaire canonique du domaine (`PARAMETERS`, `PARAMETER_UNITS`, `STAGE_ROLES`, `STAGE_LABELS`) ainsi que la forme de chaque requête/réponse.



4. Architecture de la base de données

La base de données PostgreSQL est gérée par SQLAlchemy et Alembic. Le projet utilise une migration Alembic consolidée, 0001_initial_consolidated.py, qui crée directement le schéma final de l'application. Cette migration définit l'ensemble des tables, clés étrangères, contraintes d'unicité, index et règles de suppression nécessaires au fonctionnement du système.

La migration crée notamment les tables studies, raw_sensor_data, cycles, study_cycles, cycle_plants, cycle_results, sensor_alerts, alert_config et collector_run_state.

4.1 Schéma relationnel (résumé)

```
studies
├─< raw_sensor_data      (study_id nullable, ON DELETE SET NULL)
├─< study_cycles          (study_id, ON DELETE CASCADE sur la ligne d'association uniquement)

cycles
├─< study_cycles          (cycle_id, ON DELETE CASCADE sur la ligne d'association uniquement)
├─1 cycle_plants          (cycle_id UNIQUE, ON DELETE CASCADE)
├─< cycle_results        (cycle_id, ON DELETE CASCADE)
```

Note : cycles ne possède pas de colonne study_id. Un même cycle peut être associé à plusieurs études via la table d'association study_cycles, qui constitue la source de vérité de cette relation many-to-many. sensor_alerts → studies (study_id nullable, ON DELETE SET NULL, contexte best-effort)

alert_config (ligne unique, id = 1)

collector_run_state (une ligne par set de capteurs configuré)

4.2 Tables principales

Table	Colonnes clés (non exhaustif)
studies	id, study_name, start_date, end_date
raw_sensor_data	id, study_id (nullable), created_at, entry_id, ph, temperature, ec, turbidity, do, set_number ; unique sur (entry_id, set_number)
cycles	id, cycle_name, start_date, end_date, created_at — aucune colonne study_id ; les cycles sont des entités indépendantes et réutilisables.
study_cycles	study_id, cycle_id — table d'association many-to-many ; unique sur (study_id, cycle_id) ; les deux FK sont ON DELETE CASCADE sur la ligne d'association uniquement.
cycle_plants	id, cycle_id (unique), plant_1, plant_2, plant_3
cycle_results	id, cycle_id, parameter, stage, replicate_1/2/3, average, std, removal_1/2/3, removal_percent, removal_std, stage_role
sensor_alerts	id, alert_type, sensor_set, study_id (nullable), consecutive_count, severity, status, message, created_at, updated_at, resolved_at
alert_config	id (toujours 1), null_capture_threshold
collector_run_state	set_number (PK), consecutive_empty_runs, updated_at

Choix de conception à souligner, récurrent dans les commentaires du code : les noms de plantes ne sont jamais mélangés aux lignes de résultats expérimentaux. La table cycle_results ne comporte aucune colonne « plante » ; cycle_plants est une table séparée, au plus une ligne par cycle, et la relation s'exprime uniquement

par clés étrangères — jamais via le fichier Excel importé, jamais en dupliquant un nom de plante sur les 24 lignes de résultats.

Tous les horodatages sont stockés en UTC, sous forme de colonnes `TIMESTAMP WITH TIME ZONE`. Les dates saisies via l'interface (champs `datetime-local`) ou une cellule Excel sont naïves (sans décalage UTC) : le backend interprète chaque date naïve comme étant dans le fuseau horaire `settings.TIMEZONE` (Africa/Casablanca par défaut) et la convertit en UTC avant stockage.

5. Fonctionnalités détaillées

Cette section présente chaque fonctionnalité de l'application, son fonctionnement technique, et une capture d'écran illustrant l'interface correspondante.

5.1 Gestion des études (Studies)

Une « étude » regroupe à la fois les relevés automatiques de capteurs et les cycles de traitement manuels (Cycle A, B, C, D...) pour une même campagne expérimentale. Chaque cycle nomme ses propres plantes — l'opérateur choisit une étude existante pour la poursuivre, ou en démarre une nouvelle.

Fonctionnement :

- La page d'accueil (frontend/app/page.tsx) affiche la liste des études existantes et un formulaire de création (nom, date de début, date de fin optionnelle).
- La création envoie une requête POST /api/study, gérée par app/routes/study.py et app/services/study_service.py.
- Chaque étude peut être supprimée définitivement (DELETE /api/study/{study_id}, 204 si succès, 404 si l'étude n'existe pas). Cette suppression ne cascade pas sur les cycles : les associations study_cycles sont supprimées, tandis que les cycles, leurs résultats et leurs noms de plantes sont conservés. Les données brutes de capteurs et les alertes rattachées à l'étude sont détachées (study_id mis à NULL) plutôt que supprimées.
- Deux fonctionnalités backend existent mais n'ont pas encore de page frontend dédiée : l'assignation de données brutes à une étude (POST /api/study/{id}/assign) et l'export CSV des mesures brutes d'une étude, filtré par étude (GET /api/study/{id}/export).

The screenshot shows the 'Studies' page of the Smart Water Quality System. At the top, there's a header with 'Smart Water Quality System' and 'CYCLE RESULTS'. On the right, there are links for 'Studies' and 'Alerts'. The main content area is titled 'Studies' and includes a description: 'A study groups both automatic sensor readings and manual treatment cycles (Cycle A, B, C, D...) for one experimental campaign. Each cycle names its own plants — pick a study to continue, or start a new one below.' Below this is a form to create a new study with three input fields: 'STUDY NAME' (with a placeholder 'e.g. Constructed Wetland Pilot — Summer 2026'), 'START DATE & TIME' (with a placeholder 'dd/mm/yyyy --:--' and a calendar icon), and 'END DATE & TIME (OPTIONAL)' (with a placeholder 'dd/mm/yyyy --:--' and a calendar icon). A 'New study' button is below the form. Below the form is a table of existing studies:

Study Name	Start Date & Time	End Date & Time	Actions
Summer 2026	21/02/2026 00:00	01/10/2026 00:00	
Summer 2025	01/01/2025 00:00	01/10/2025 00:00	

Figure 1 — Page « Studies » : liste des études et formulaire de création d'une nouvelle étude.

5.2 Gestion des cycles de laboratoire

À l'intérieur d'une étude, l'opérateur gère une liste de cycles de traitement (Cycle A, B, C, D...), chacun avec ses propres dates de début et de fin et son propre jeu de résultats sur 24 lignes (8 paramètres de qualité de l'eau × 3 étapes de traitement).

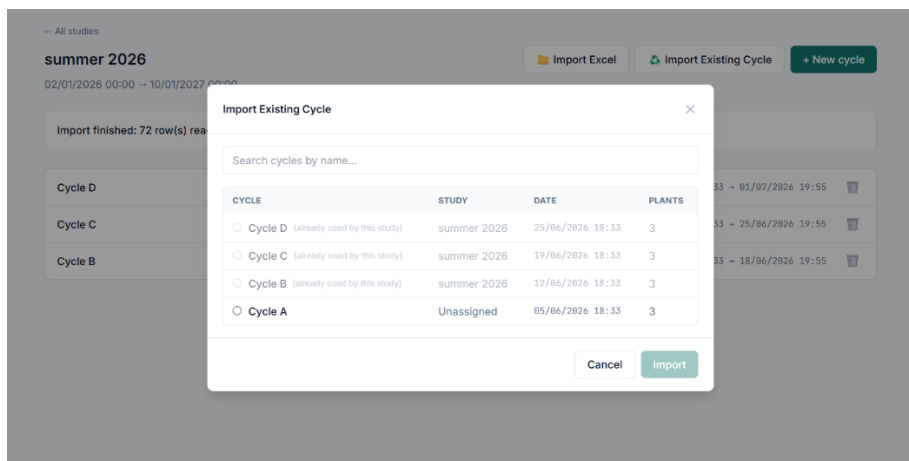
Fonctionnement :

- La page `/studies/{studyId}` (`frontend/app/studies/[studyId]/page.tsx`) liste les cycles de l'étude, avec un bouton « + New cycle » pour la saisie manuelle et un bouton « Import Excel » pour l'import en masse.
- La création manuelle d'un cycle appelle POST `/api/cycles` avec les 24 lignes de mesures plus les noms de plantes, construits côté client.
- Un cycle n'est en revanche JAMAIS supprimable depuis l'application : il n'existe intentionnellement aucun endpoint DELETE `/api/cycles/{cycle_id}`. Seule l'association entre un cycle et une étude peut être retirée ; le cycle, ses résultats et ses noms de plantes restent conservés.

IMPORTANT :

Réutilisation d'un cycle existant entre plusieurs études

- Avec la table d'association `study_cycles`, un même cycle peut être rattaché à plusieurs études à la fois, sans être dupliqué ni recréé. Seule la ligne d'association est créée ou supprimée.
- Le bouton « Import Existing Cycle » de la page d'une étude (composant `frontend/components/ImportExistingCycleModal.tsx`) ouvre une fenêtre listant TOUS les cycles de la base de données (GET `/api/cycles/available-for-import`, avec une recherche côté client sur le nom du cycle), en indiquant pour chacun les études qui l'utilisent déjà.



- Choisir un cycle envoie POST `/api/study/{study_id}/cycles/{cycle_id}/import`, qui crée uniquement une nouvelle ligne `study_cycles` (erreur 409 si le cycle est déjà associé à cette étude) — aucun fichier n'est envoyé, aucune donnée du cycle (résultats, plantes) n'est copiée ou clonée.
- À l'inverse, DELETE `/api/study/{study_id}/cycles/{cycle_id}` retire un cycle d'une étude sans le supprimer : le cycle, ses résultats et ses noms de plantes restent intacts et disponibles pour toute autre étude (ou pour être ré-importés plus tard dans la même étude).
- Cette fonctionnalité permet, par exemple, de comparer plusieurs campagnes expérimentales qui partagent un cycle de contrôle commun, sans jamais ressaisir ni dupliquer les mesures de ce cycle.

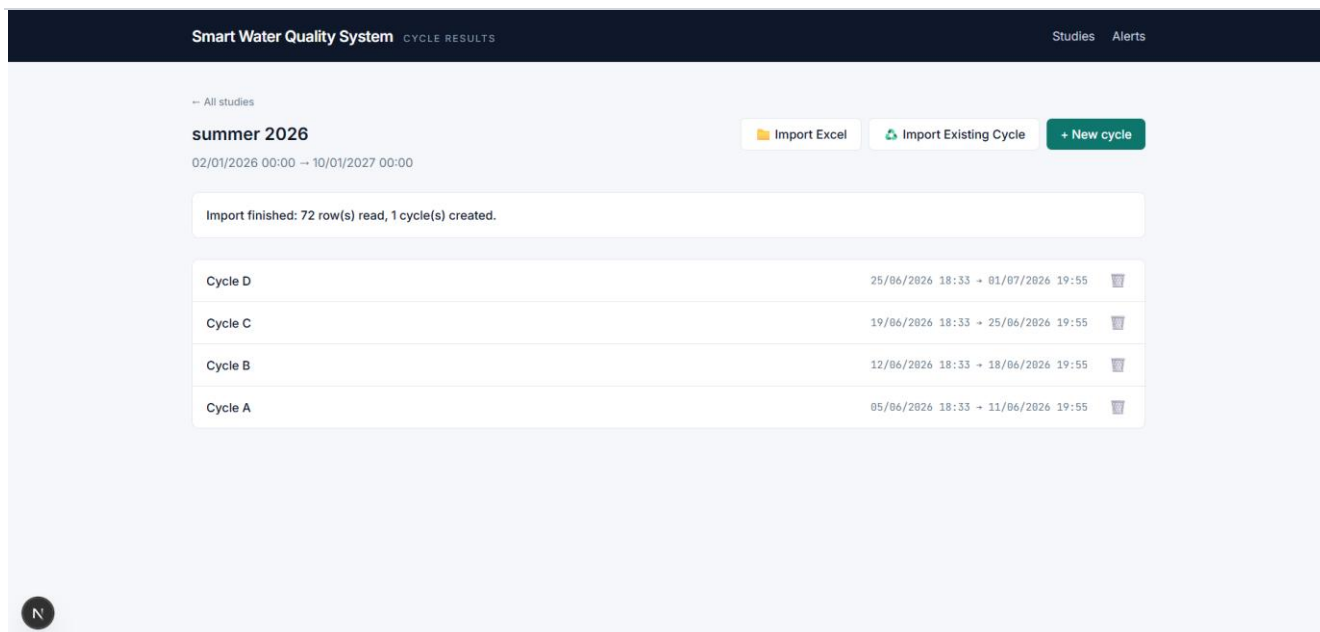


Figure 2 — Liste des cycles d'une étude (Cycle A à D), avec les actions « Import Excel » et «import existing cycle» et « + New cycle ».

5.3 Résultats de cycle et calculs statistiques

Le cœur métier de l'application : pour chaque cycle, 8 paramètres de qualité de l'eau (DCO/COD, DBO/BOD, MES/TSS, pH, etc.) sont mesurés à 3 étapes — eau brute (Wastewater, l'étape de référence/baseline), série plantée (Planted Series) et série témoin (Control Series) — chacune en triplicat (3 réplicats).

Pour chaque ligne (paramètre, étape), le backend calcule automatiquement :

- La moyenne (average) des 3 réplicats.
- L'écart-type d'échantillon (std).
- Pour les étapes non-baseline (planted / control) : le pourcentage d'abattement (removal %) par rapport à l'étape « Wastewater » du même paramètre, ainsi que l'écart-type de cet abattement (removal_std).

Ces calculs sont effectués de manière authoritative côté backend (backend/app/services/calculations.py, fonction `build_cycle_results()`), et dupliqués de façon contrôlée côté frontend (frontend/lib/calculations.ts) uniquement pour donner un aperçu instantané pendant la saisie — les deux implémentations ne peuvent différer que par l'arrondi d'affichage. Le formulaire de saisie recalcule ainsi les valeurs en direct pendant que l'opérateur tape, avant l'enregistrement définitif qui recharge les valeurs calculées par le serveur.

Chaque cycle porte également jusqu'à trois noms de plantes (Plant 1, Plant 2, Plant 3), stockés séparément dans la table `cycle_plants` — ce nom n'est jamais copié dans les 24 lignes de résultats elles-mêmes, garantissant que l'identité de la plante reste une donnée indépendante des mesures.

Smart Water Quality System CYCLE RESULTS Studies Alerts

← Back to study

Cycle D 25/06/2026 18:33 - 01/07/2026 19:55
Stored results as calculated by the backend at save time.

Plant names for this cycle
Plant 1 Mint Plant 2 Oregano Plant 3 Thyme

PARAMETER	STAGE	REPLICATE 1	REPLICATE 2	REPLICATE 3	AVERAGE	STD	REMOVAL 1	REMOVAL 2	REMOVAL 3	REMOVAL (%)	REMOVAL STD
COD (mg/L)	Wastewater	315	300	310	308.33	7.64	n/a	n/a	n/a	n/a	n/a
	Planted Series	100	90	95	95.00	5.00	68.3	70.0	69.4	69.2%	0.88
	Control Series	135	125	130	130.00	5.00	57.1	58.3	58.1	57.8%	0.62
BOD (mg/L)	Wastewater	250	235	245	243.33	7.64	n/a	n/a	n/a	n/a	n/a
	Planted Series	38	35	36	36.33	1.53	84.8	85.1	85.3	85.1%	0.25
	Control Series	55	50	52	52.33	2.52	78.0	78.7	78.8	78.5%	0.43
TSS (mg/L)	Wastewater	2.45	2.15	2.35	2.32	0.15	n/a	n/a	n/a	n/a	n/a
	Planted Series	0.38	0.32	0.35	0.35	0.03	84.5	85.1	85.1	84.9%	0.36
	Control Series	0.52	0.46	0.48	0.49	0.03	78.8	78.6	79.6	79.8%	0.52
pH	Wastewater	7.1	6.8	7	6.97	0.15	n/a	n/a	n/a	n/a	n/a
	Planted Series	6	5.8	5.9	5.90	0.10	15.5	14.7	15.7	15.3%	0.53
	Control Series	6.2	6	6.1	6.10	0.10	12.7	11.8	12.9	12.4%	0.59

Figure 3 — Détail d'un cycle (Cycle D) : noms des plantes et tableau des résultats (réplicats, moyenne, écart-type, abattement %) pour COD, BOD, TSS et pH.

5.4 Import de cycles depuis Excel

Pour éviter la ressaisie manuelle de nombreux cycles, l'application permet d'importer un classeur Excel (.xlsx / .xls) contenant plusieurs cycles à la fois.

- Le bouton « Import Excel » (visible sur la page d'une étude) envoie le fichier vers POST /api/study/{id}/cycles/import (requête multipart).
- Le fichier est analysé côté serveur avec pandas.read_excel (backend/app/services/cycle_import_service.py), regroupé par cycle, puis chaque groupe de lignes est validé et transformé en triplets (paramètre, étape, réplicats).
- Les lignes invalides sont collectées comme erreurs (row_errors) sans interrompre le reste de l'import ; les cycles dont le nom existe déjà pour l'étude sont ignorés par défaut.
- Le même calcul de statistiques (build_cycle_results()) est appliqué que la donnée provienne du formulaire manuel ou du fichier Excel — garantissant une cohérence totale entre les deux voies de saisie.
- Le frontend affiche ensuite un résumé de l'import (nombre de lignes lues, cycles créés, cycles ignorés avec leur raison, erreurs de ligne) et rafraîchit la liste des cycles.
- Seul le type de fichier est vérifié avant analyse (extension .xlsx/.xls), sans authentification requise sur l'endpoint.

Cycle Name	Start Date	End Date	Parameter	Stage	Replicate	Value	Plant 1	Plant 2	Plant 3
Cycle A	05/06/2026 18:33	11/06/2026 19:55	COD	Wastewater	1	310	Tomato	Lettuce	Mint
				Wastewater	2	295			
				Wastewater	3	316			
				Planted Series	1	160			
				Planted Series	2	148			
				Planted Series	3	155			
				Control Series	1	190			
				Control Series	2	178			
				Control Series	3	185			
			BOD	Wastewater	1	245			
				Wastewater	2	230			
				Wastewater	3	245			
				Planted Series	1	72			
				Planted Series	2	65			
				Planted Series	3	68			
				Control Series	1	92			
				Control Series	2	85			
				Control Series	3	88			
			TSS	Wastewater	1	2.5			
				Wastewater	2	2.1			
				Wastewater	3	2.42			
				Planted Series	1	0.68			
				Planted Series	2	0.6			
				Planted Series	3	0.64			
				Control Series	1	0.85			
				Control Series	2	0.78			
				Control Series	3	0.82			
			pH	Wastewater	1	7.2			
				Wastewater	2	6.9			
				Wastewater	3	7.1			
				Planted Series	1	6.5			
				Planted Series	2	6.3			

Figure 4 — exemple de format exact du fichier Excel

5.5 Collecte automatique des données de capteurs (ThingSpeak)

En parallèle du flux manuel de cycles, le backend collecte en continu les mesures brutes provenant de canaux ThingSpeak (jusqu'à N « sets de capteurs » configurés).

- Un planificateur APScheduler déclenche `scheduled_collection_job()` toutes les `COLLECTOR_INTERVAL_MINUTES` minutes (et une fois immédiatement au démarrage du backend).
- Séparément de cette collecte récurrente, une purge historique complète (`backfill_all()`, exécutée UNE SEULE FOIS au démarrage du backend, avant même la première collecte régulière) récupère l'intégralité de l'historique disponible sur chaque canal ThingSpeak configuré — potentiellement des dizaines de milliers d'entrées — via une pagination inversée (paramètre `end` reculant progressivement dans le temps) qui contourne une limite de l'API ThingSpeak sur les plages `start/end` trop larges ; ce mécanisme est décrit en détail au §8.5.3.
- Pour chaque set de capteurs configuré, `ThingSpeakCollector.run()` récupère les flux récents depuis l'API ThingSpeak, les nettoie et les normalise via un `DataFrame` pandas (renommage des champs `field1..field5` en `ph/temperature/ec/turbidity/do`, conversion de types, suppression des lignes sans horodatage ou sans identifiant d'entrée).
- Les nouvelles lignes sont insérées dans `raw_sensor_data` via `INSERT ... ON CONFLICT DO NOTHING`, avec une clé unique sur (`entry_id`, `set_number`) — le mécanisme est donc idempotent par construction : relancer une collecte n'introduit jamais de doublons.
- Une collecte manuelle est également possible via l'endpoint `POST /api/collect`.
- Les données brutes peuvent, ultérieurement et facultativement, être rattachées à une étude via `POST /api/study/{id}/assign` — une simple mise à jour sur les lignes de la plage de dates de l'étude qui ne sont pas encore assignées, sans copie ni duplication de données.

5.6 Détection et gestion des alertes

Le système surveille en permanence la santé du pipeline de collecte de capteurs et alerte l'opérateur sans qu'une surveillance manuelle ne soit nécessaire. Trois catégories indépendantes sont suivies :

- No Data Collected (aucune donnée collectée) : aucun nouvel enregistrement n'a été reçu de ThingSpeak pour N tentatives de collecte consécutives.
- Capture Failure (échec de capture) : un ou plusieurs sets de capteurs ont renvoyé uniquement des valeurs à zéro pour N captures consécutives — un capteur est considéré en échec seulement si ses cinq valeurs de paramètres sont toutes exactement égales à 0.
- Anomaly Detection (détection d'anomalie) : réservée aux futures violations de seuils de capteurs configurables ; le type existe dans le schéma et dans l'interface, mais la fonction `detect_anomalies()` ne renvoie actuellement jamais d'anomalie (fonctionnalité non implémentée, réservée pour une évolution future).

Le seuil N (nombre d'échecs consécutifs déclenchant une alerte) est configurable depuis la page « Alerts » (champ « Consecutive failed captures threshold », bouton « Save Threshold »), lu et modifié via `GET/PUT /api/alerts/config`.

Cycle de vie d'une alerte :

- Chaque exécution du collecteur évalue indépendamment `no_data` (par set) et `capture_failure` (par set, plus un cas combiné « tous les sets à la fois »), en comparant un compteur d'échecs consécutifs en cours au seuil configuré N.
- Dès que le compteur atteint N, une ligne d'alerte est créée avec le statut « active » et la sévérité « warning » ; si le compteur continue de croître au-delà de N, la sévérité passe à « critical ».
- Uniquement lors du passage à une nouvelle alerte active, `notify_new_alert()` déclenche les canaux e-mail et Telegram (chacun indépendamment optionnel, configuré par variables d'environnement).
- Dès que la condition sous-jacente disparaît (arrivée de nouvelles données ; apparition d'une capture avec une valeur non nulle), l'alerte active est marquée « resolved » avec un horodatage `resolved_at`.

La page « Alerts » offre un tableau filtrable (par type d'alerte, statut, sévérité, set de capteurs) affichant pour chaque alerte : sa description, le set de capteurs concerné, la sévérité, le statut, le nombre d'échecs consécutifs, l'heure de détection et l'heure de résolution.

Alerts

Alerts fire after N consecutive failed events. Three independent categories are tracked: missing collections, all-zero captures, and (in the future) sensor-value anomalies.

CONSECUTIVE FAILED CAPTURES THRESHOLD (N)
 Save Threshold Currently: Alert after 3 consecutive failed captures.

No Data Collected
 No new records have been received from ThingSpeak for N consecutive collection attempts.

Capture Failure
 One or more sensor parameter sets returned only zero values for N consecutive captures.

Anomaly Detection (Future)
 Reserved for configurable sensor threshold violations.

All alert types All statuses All severities All sensor sets

ALERT TYPE	DESCRIPTION	SENSOR SET	SEVERITY	STATUS	CONSECUTIVE FAILURES	DETECTION TIME	RESOLUTION TIME
No Data Collected	SET 2 : no new records received from ThingSpeak for 6 consecutive collection attempts.	Set 2	CRITICAL	ACTIVE	6	08/09/2026 17:01	—
No Data Collected	SET 2 : no new records received from ThingSpeak for 120 consecutive collection attempts.	Set 2	CRITICAL	RESOLVED	120	07/09/2026 16:48	08/09/2026 11:25

Figure 4 — Page « Alerts » : configuration du seuil, description des trois catégories d'alertes, et tableau filtrable des alertes détectées.

5.7 Notifications par e-mail et Telegram

backend/app/services/notification_service.py envoie une notification (e-mail via smtplib, message via l'API Bot Telegram) uniquement lorsqu'une alerte devient nouvellement active — pas à chaque évaluation périodique. Chaque canal est indépendamment optionnel et configuré par variables d'environnement (identifiants SMTP, jeton de bot et identifiant de discussion Telegram). Un échec d'envoi (e-mail ou Telegram) est intercepté, journalisé, et jamais propagé : il ne peut jamais interrompre le pipeline de détection d'alertes ni l'exécution du collecteur.

[Water Quality] Alert: no_data (set 2) Boîte de réception x

elboumashouli.achraf@etu.uae.ac.ma
 À moi ▾

Sensor alert triggered

Type: no_data
 Sensor set: 2
 Severity: warning
 Consecutive failures: 3
 Study ID: n/a
 Created at (UTC): 2026-09-08 17:01:44.580303+01:00

SET 2 : no new records received from ThingSpeak for 3 consecutive collection attempts.

Répondre Transférer Partager dans le chat

Smart Water Quality System Alerts bot

Sensor alert triggered

Type: no_data
 Sensor set: 2
 Severity: warning
 Consecutive failures: 3
 Study ID: n/a
 Created at (UTC): 2026-08-21 03:34:15.443254+01:00

SET 2 : no new records received from ThingSpeak for 3 consecutive collection attempts. 03:34

September 4

Sensor alert triggered

Type: no_data
 Sensor set: 2
 Severity: warning
 Consecutive failures: 3

Write a message...

5.8 API REST (documentation interactive)

L'API est une API REST/JSON conventionnelle, sans préfixe de version au-delà de /api. La documentation interactive est générée automatiquement par FastAPI et accessible aux chemins /docs (Swagger UI) et /redoc.

Ressource	Chemin de base	Fichier de routes
Santé / méta	/, /api/sensor-sets, /api/scheduler, /api/collect	app/main.py
Données brutes de capteurs	/api/raw-data	app/routes/raw_data.py
Études	/api/study	app/routes/study.py
Cycles	/api/cycles, /api/meta	app/routes/cycle.py
Alertes	/api/alerts	app/routes/alerts.py

Les corps de requête/réponse sont validés par des schémas Pydantic ; les erreurs de validation apparaissent comme un 422 standard FastAPI, les erreurs métier comme des exceptions HTTP 400/404 levées explicitement dans les gestionnaires de routes, et toute erreur imprévue comme un 500 générique renvoyé par le gestionnaire d'exceptions global.

6. Configuration de l'environnement avec le fichier .env

Afin de rendre l'application flexible et facilement configurable, les paramètres sensibles ou susceptibles de changer selon l'environnement d'exécution sont externalisés dans des fichiers de configuration .env.

Cette approche permet de modifier la configuration de la base de données, des capteurs, du planificateur de collecte, du fuseau horaire et des systèmes de notification sans modifier le code source de l'application.

Le backend utilise le fichier backend/.env, tandis que le frontend utilise frontend/.env.local.

1. Configuration du backend

Le fichier backend/.env contient les paramètres nécessaires au fonctionnement de l'API FastAPI et du système de collecte.

Base de données PostgreSQL

DATABASE_URL=postgresql://postgres:<password>@localhost:5432/water_quality_db

Cette variable définit la chaîne de connexion à PostgreSQL. Elle contient :

le type de base de données (postgresql) ;

le nom d'utilisateur ;

le mot de passe ;

l'adresse du serveur ;

le port PostgreSQL (5432) ;

le nom de la base de données (water_quality_db).

L'utilisation d'une variable d'environnement permet de changer de base de données sans modifier le code de l'application.

2. Configuration générale de l'application

APP_NAME=Water Quality Monitoring API

LOG_LEVEL=INFO

APP_NAME définit le nom de l'application backend.

LOG_LEVEL contrôle le niveau de détail des journaux générés par l'application. Par exemple, la valeur INFO permet de conserver les informations générales sur l'exécution du système.

3. Configuration des capteurs ThingSpeak

Les canaux ThingSpeak utilisés pour récupérer les données des capteurs sont configurés à travers la variable SENSOR_SETS.

```
SENSOR_SETS=[  
  {"set_number": 1, "channel_id": "3111290", "api_key": null},  
  {"set_number": 2, "channel_id": "3243723", "api_key": null}  
]
```

Chaque ensemble de capteurs est défini par :

set_number : numéro identifiant l'ensemble de capteurs ;

channel_id : identifiant du channel ThingSpeak ;

api_key : clé API de lecture lorsque le channel est privé.

Cette configuration est particulièrement importante car elle permet d'ajouter, supprimer ou modifier des ensembles de capteurs sans effectuer de modification dans le code source.

La configuration est représentée sous forme JSON et est validée automatiquement au démarrage de l'application afin d'éviter une configuration invalide.

4. Configuration de la collecte automatique

COLLECTOR_INTERVAL_MINUTES=5

THINGSPEAK_RESULTS_COUNT=10

COLLECTOR_INTERVAL_MINUTES définit l'intervalle entre deux opérations de collecte des données depuis ThingSpeak. Dans la configuration actuelle, une collecte est effectuée toutes les 5 minutes.

THINGSPEAK_RESULTS_COUNT définit le nombre de mesures récupérées lors d'un appel à l'API ThingSpeak.

Cette configuration est utilisée par le scheduler basé sur APScheduler, permettant au système de fonctionner automatiquement sans intervention manuelle.

5. Configuration du fuseau horaire

TIMEZONE=Africa/Casablanca

Le paramètre TIMEZONE définit le fuseau horaire métier utilisé par l'application.

Le système utilise ce paramètre pour :

les formulaires de saisie ;

l'importation des données Excel ;

les logs ;

le scheduler ;

l'affichage des dates et heures.

Les données temporelles sont conservées en base de données en UTC, tandis que les conversions entre UTC et le fuseau horaire utilisé par l'application sont réalisées lors de la saisie et de l'affichage.

Ainsi, la modification de cette seule variable permet d'adapter le système à un autre fuseau horaire sans modifier la logique métier.

6. Configuration du système d'alertes par e-mail

Le système permet d'envoyer automatiquement des notifications par e-mail lorsqu'une alerte est déclenchée.

Les paramètres sont configurés comme suit :

SMTP_HOST=

SMTP_PORT=587

SMTP_USERNAME=

SMTP_PASSWORD=

SMTP_FROM_EMAIL=

SMTP_USE_TLS=true

ALERT_EMAIL_RECIPIENTS=

Les principaux paramètres sont :

SMTP_HOST : serveur SMTP utilisé pour l'envoi ;

SMTP_PORT : port du serveur SMTP ;

SMTP_USERNAME : identifiant SMTP ;

SMTP_PASSWORD : mot de passe SMTP ;

SMTP_FROM_EMAIL : adresse utilisée comme expéditeur ;

SMTP_USE_TLS : activation de la sécurisation TLS ;

`ALERT_EMAIL_RECIPIENTS` : liste des destinataires.

Les destinataires peuvent être séparés par des virgules :

`ALERT_EMAIL_RECIPIENTS=admin@example.com,operator@example.com`

Le système considère le service e-mail comme optionnel. Si les paramètres nécessaires ne sont pas configurés, les notifications e-mail sont simplement désactivées sans provoquer d'erreur bloquante dans l'application.

7. Configuration des notifications Telegram

Le système prend également en charge l'envoi de notifications via Telegram.

`TELEGRAM_BOT_TOKEN=`

`TELEGRAM_CHAT_ID=`

`TELEGRAM_BOT_TOKEN` correspond au token du bot Telegram.

`TELEGRAM_CHAT_ID` identifie le destinataire des notifications, qui peut être un utilisateur, un groupe ou un canal.

Les notifications Telegram ne sont activées que lorsque les deux paramètres sont correctement configurés. Dans le cas contraire, le système continue son fonctionnement normalement et les notifications Telegram sont simplement ignorées.

8. Seuil de détection des alertes

Le nombre de captures consécutives invalides ou NULL nécessaires avant de déclencher une alerte peut également être configuré :

`NULL_CAPTURE_THRESHOLD=3`

Dans cet exemple, une alerte est générée après 3 captures consécutives NULL pour un capteur et un paramètre donné.

Ce paramètre peut être défini dans `.env` et peut également être modifié dynamiquement via la configuration des alertes enregistrée dans la base de données.

9. Configuration du frontend

Le frontend Next.js possède son propre fichier `.env.local`.

`NEXT_PUBLIC_API_URL=http://localhost:8000`

`NEXT_PUBLIC_TIMEZONE=Africa/Casablanca`

`NEXT_PUBLIC_API_URL` définit l'adresse de l'API backend à laquelle le frontend envoie ses requêtes.

Par défaut :

`http://localhost:8000`

`NEXT_PUBLIC_TIMEZONE` définit le fuseau horaire utilisé par l'interface pour afficher les dates et heures.

Il est important que NEXT_PUBLIC_TIMEZONE corresponde au paramètre TIMEZONE du backend afin d'assurer une cohérence entre les dates saisies, stockées et affichées.

10. Avantages de cette approche

La centralisation de la configuration dans les fichiers .env apporte plusieurs avantages :

Flexibilité : les paramètres peuvent être modifiés sans toucher au code source.

Séparation configuration/code : les informations liées à l'environnement sont séparées de la logique métier.

Facilité de déploiement : la même application peut être utilisée en développement, test ou production avec des configurations différentes.

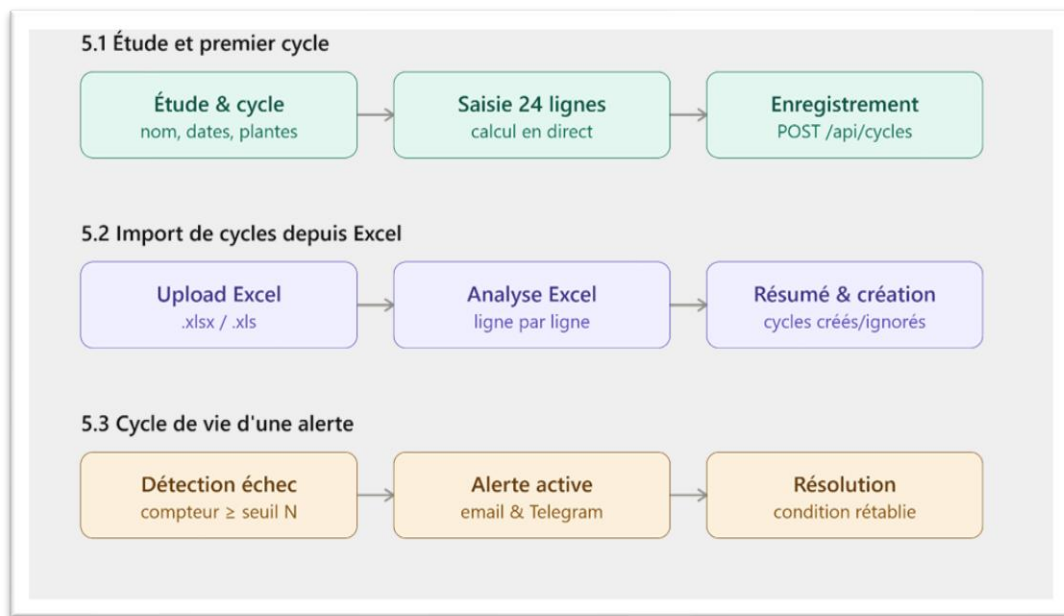
Sécurité : les informations sensibles, telles que les mots de passe SMTP ou les clés API, peuvent être conservées dans l'environnement plutôt que directement dans le code.

Évolutivité : de nouveaux channels ThingSpeak ou de nouveaux paramètres peuvent être ajoutés simplement en modifiant la configuration.

Maintenance simplifiée : les paramètres principaux du système sont regroupés dans un emplacement clairement identifié.

En résumé, le système adopte une configuration centralisée et externalisée, ce qui facilite son déploiement, sa maintenance et son adaptation à différents environnements.

7. Flux de travail principaux



7.1 Création d'une étude et de son premier cycle

- 1. L'opérateur ouvre la page d'accueil, saisit le nom de l'étude + date de début (+ date de fin optionnelle), valide → POST /api/study.
- 2. L'opérateur ouvre la nouvelle étude (/studies/{id}), puis clique sur « + New cycle ».
- 3. L'opérateur nomme le cycle, définit sa plage de dates, renseigne facultativement jusqu'à trois noms de plantes, et saisit les valeurs de réplicats pour autant des 24 cellules (paramètre, étape) que disponibles ; le tableau recalcule moyenne/écart-type/abattement en direct dans le navigateur.
- 4. Sur « Save cycle », l'intégralité du payload est envoyée en une seule requête POST /api/cycles ; le backend valide que les 24 combinaisons sont présentes, calcule les statistiques autoritatives, et persiste l'ensemble.
- 5. L'opérateur est redirigé vers la page de détail du cycle, qui recharge les valeurs enregistrées et calculées par le backend via GET /api/cycles/{id}.

Réutiliser un cycle existant dans une autre étude

- Depuis une étude différente, l'opérateur clique sur « Import Existing Cycle », recherche le cycle par son nom dans la liste complète renvoyée par GET /api/cycles/available-for-import, et le sélectionne.
- Le frontend envoie POST /api/study/{study_id}/cycles/{cycle_id}/import, qui crée uniquement une nouvelle ligne study_cycles — aucune donnée du cycle n'est recopiée, et le cycle reste simultanément visible dans toutes les études qui l'utilisent.
- Retirer ce même cycle d'une étude (sans le supprimer nulle part) se fait via DELETE /api/study/{study_id}/cycles/{cycle_id}, qui supprime uniquement cette ligne study_cycles.

7.2 Import de cycles depuis Excel

- 1. Depuis la page d'une étude, l'opérateur clique sur « Import Excel » et sélectionne un fichier .xlsx/.xls.

- 2. Le fichier est analysé côté serveur (`pandas.read_excel`), regroupé par cycle, puis validé ligne par ligne ; les lignes invalides sont collectées comme `row_errors` sans stopper le reste de l'import ; les cycles dont le nom existe déjà pour l'étude sont ignorés par défaut.
- 3. Le frontend affiche un résumé (lignes lues, cycles créés, cycles ignorés avec raisons, erreurs de ligne) et rafraîchit la liste des cycles en place.

7.3 Cycle de vie d'une alerte

- 1. Chaque exécution du collecteur évalue `no_data` (par set) et `capture_failure` (par set, plus un cas combiné) indépendamment, en comparant un compteur d'échecs consécutifs au seuil configuré N.
- 2. Une fois le compteur à N, une alerte est créée en statut « active » et sévérité « warning » ; au-delà de N, elle devient « critical ».
- 3. Uniquement lors du passage à une nouvelle alerte active, les canaux e-mail et Telegram sont déclenchés.
- 4. Une fois la condition sous-jacente résolue, l'alerte est marquée « resolved » avec un horodatage.
- 5. Les opérateurs consultent/filtrent cet historique et ajustent le seuil partagé depuis la page /alerts du frontend.

8. Guide technique et théorique

Cette seconde partie du rapport est un guide de référence détaillé, construit directement à partir du code source (backend Python/FastAPI et frontend Next.js/TypeScript) et non uniquement des README. Elle vise à permettre à un développeur qui découvre le projet de comprendre le POURQUOI de chaque choix technique, de localiser précisément où intervenir, et de modifier le code en toute sécurité sans casser les garanties existantes (idempotence, séparation plantes/résultats, cohérence des calculs, etc.).

Convention de lecture : chaque sous-section indique le ou les fichiers exacts concernés, explique la théorie/le concept sous-jacent, puis détaille le fonctionnement réel du code avec ses cas limites.

8.1 Théorie et implémentation des calculs statistiques

Fichiers concernés : backend/app/services/calculations.py (source de vérité), frontend/lib/calculations.ts (miroir pour aperçu instantané).

8.1.1 Moyenne (average)

Pour chaque ligne (paramètre, étape), la moyenne arithmétique est calculée sur les réplicats renseignés et non manquants (NULL) uniquement. La valeur 0 est considérée comme une valeur valide et est incluse dans le calcul :

$$\text{moyenne} = (\sum \text{réplicats renseignés}) / (\text{nombre de réplicats renseignés})$$

Implémentation : compute_average() filtre d'abord les valeurs None via _clean(), puis utilise statistics.mean(). Si aucun réplicat n'est renseigné, la fonction retourne None (jamais 0, jamais une erreur) — le frontend affiche alors « n/a ».

8.1.2 Écart-type d'échantillon (std)

L'application utilise systématiquement l'écart-type D'ÉCHANTILLON (ddof=1), c'est-à-dire la même formule que la fonction STDEV d'Excel — et non l'écart-type de population (ddof=0, STDEVP dans Excel) :

$$\text{écart-type} = \sqrt{(\sum (x_i - \text{moyenne})^2 / (n - 1))}$$

Implémentation : compute_std() appelle statistics.stdev() (Python), qui utilise n-1 au dénominateur. Elle retourne explicitement None si moins de 2 valeurs sont disponibles (l'écart-type n'est mathématiquement pas défini pour un seul point). Côté frontend, sampleStd() dans lib/calculations.ts reproduit exactement la même formule en JavaScript pur, sans dépendance externe.

Point d'attention pour toute modification : si vous changez cette formule côté backend (par exemple pour passer à un écart-type de population), vous DEVEZ répercuter le changement dans frontend/lib/calculations.ts, sinon l'aperçu en direct du formulaire de saisie divergera silencieusement des valeurs enregistrées par le serveur.

8.1.3 Pourcentage d'abattement (Removal %)

Le Removal % mesure la réduction d'un paramètre entre l'échantillon brut (baseline, stage_1 = « Wastewater ») et un échantillon traité (stage_2 « Planted Series » ou stage_3 « Control Series »), du MÊME paramètre :

$$\text{removal (\%)} = ((\text{valeur_baseline} - \text{valeur_traitée}) / \text{valeur_baseline}) \times 100$$

Ce calcul est effectué réplicat par réplicat (removal_1, removal_2, removal_3), puis le removal_percent final est la MOYENNE de ces trois valeurs de removal (et non un removal calculé sur les moyennes globales) — et removal_std est l'écart-type d'échantillon de ces trois mêmes valeurs de removal.

Cas limites gérés explicitement par compute_removal() :

- Si la valeur baseline ou la valeur courante est None → le removal de ce réplicat est None (pas d'erreur, pas de 0 par défaut).
- Si la valeur baseline vaut exactement 0 → le removal est None (protection division par zéro) plutôt que de lever une exception ou de retourner l'infini.
- Un removal négatif est mathématiquement valide et signifie que le traitement a AUGMENTÉ la concentration du paramètre par rapport au brut (ex. -15.3% pour le pH dans l'exemple Cycle D du rapport) — l'interface l'affiche en rouge/orange (RemovalBadge.tsx) plutôt que de le masquer.

8.1.4 Arrondi et précision d'affichage

round_or_none() arrondit chaque valeur calculée à 4 décimales avant persistance (backend) ; round4() fait de même à 4 décimales côté frontend pour l'aperçu. Cet arrondi commun garantit que la seule source de divergence possible entre les deux implémentations est un arrondi d'affichage, jamais un écart de méthode de calcul.

8.2 Modèle de données détaillé (théorie relationnelle + implémentation SQLAlchemy)

Fichiers concernés : backend/app/models/*.py (dont models/study_cycle.py) et backend/alembic/versions/0001_initial_consolidated.py.

8.2.1 Principe directeur : séparation stricte plantes / résultats

C'est la décision de conception la plus structurante du projet, répétée dans presque tous les commentaires du code : un nom de plante n'est JAMAIS une colonne de cycle_results, jamais dupliqué sur les 24 lignes de résultats, et jamais porté par le fichier Excel comme une relation avec un résultat particulier.

Justification théorique : les noms de plantes sont un attribut du CYCLE dans son ensemble (« quelle espèce a été testée durant cette campagne »), pas un attribut de chaque MESURE. Les modéliser comme deux entités distinctes reliées par clé étrangère (plutôt que de dupliquer le nom de plante sur chacune des 24 lignes) élimine par construction tout risque d'incohérence (ex. deux lignes du même cycle portant des noms de plantes différents), au prix d'une jointure supplémentaire à la lecture — un compromis normal en modélisation relationnelle (normalisation 3NF).

- cycle_plants.cycle_id → cycles.id (relation 1-1, contrainte UNIQUE sur cycle_id, ON DELETE CASCADE)
- cycle_results.cycle_id → cycles.id (relation 1-N, ON DELETE CASCADE)
- Aucune clé étrangère ni colonne ne relie directement cycle_plants et cycle_results : leur seul point commun est cycles.id.

8.2.2 ON DELETE CASCADE vs ON DELETE SET NULL — pourquoi ce choix diffère selon la table

Deux stratégies de suppression coexistent délibérément dans le schéma :

- CASCADE (cycles → cycle_results, cycles → cycle_plants, studies → study_cycles, cycles → study_cycles) : ces données n'ont aucun sens sans leur parent — un cycle_result orphelin (sans cycle) serait une donnée

incohérente, tout comme une ligne `study_cycles` pointant vers une étude ou un cycle qui n'existe plus. Le CASCADE sur `study_cycles` ne s'applique JAMAIS qu'à la ligne d'association elle-même : supprimer une étude retire ses lignes `study_cycles`, mais ne touche ni ne cascade jamais jusqu'aux cycles, `cycle_results` ou `cycle_plants` qu'elle référençait (voir §8.2.4 ci-dessous).

- SET NULL (`studies` → `raw_sensor_data`, `studies` → `sensor_alerts`) : les mesures brutes de capteurs ont une existence légitime indépendamment de toute étude (elles sont collectées en continu, avant même qu'un opérateur ne les rattache à une étude). Supprimer une étude ne doit donc jamais détruire des mesures de capteurs déjà collectées — seul le lien (`study_id`) est retiré.

Implémentation SQLAlchemy : ce comportement est déclaré à DEUX niveaux qui doivent rester cohérents entre eux : (1) au niveau base de données, via `ForeignKey(..., ondelete="CASCADE"/"SET NULL")` dans les modèles, qui génère la contrainte SQL réelle ; (2) au niveau ORM Python, via `cascade="all, delete-orphan"`, `passive_deletes=True` sur les `relationship()` correspondantes dans `app/models/cycle.py`, pour que SQLAlchemy laisse PostgreSQL faire le travail plutôt que d'émettre un DELETE ligne par ligne en Python (`passive_deletes=True` est essentiel : sans ce paramètre, SQLAlchemy ignorerait la contrainte SQL et tenterait de charger puis supprimer chaque ligne enfant individuellement, ce qui serait à la fois plus lent et redondant avec la contrainte déjà en place).

8.2.3 Rôles d'étape stables vs libellés d'affichage

Un autre choix de conception récurrent : chaque ligne de résultat porte DEUX représentations de son étape expérimentale : `stage_role` (« `stage_1` »/« `stage_2` »/« `stage_3` », un identifiant STABLE utilisé en interne pour retrouver la ligne baseline lors du calcul de Removal %) et `stage` (le libellé réel affiché à l'utilisateur : « Wastewater »/« Planted Series »/« Control Series », résolu UNE SEULE FOIS à l'écriture via `STAGE_LABELS` et persisté tel quel en base).

Pourquoi ne pas dériver `stage` depuis `stage_role` à chaque lecture ? Parce que cela rendrait ce libellé implicitement recalculé (et donc fragile si `STAGE_LABELS` change un jour), alors que le persister directement garantit que la base de données contient la valeur définitive telle qu'elle a été enregistrée. `CycleResultOut` porte néanmoins un `model_validator` (`resolve_stage_label`) qui ne fait que sécuriser le cas où une ligne existante n'aurait, pour une raison quelconque, pas encore de valeur `stage` — un filet de sécurité, pas le mécanisme principal.

8.2.4 La table d'association `study_cycles` : relation many-to-many

La relation entre `Study` et `Cycle` est directement modélisée comme une relation **many-to-many** au niveau du schéma final. La table `study_cycles` contient les clés étrangères `study_id` et `cycle_id`, ainsi qu'une contrainte d'unicité sur le couple (`study_id`, `cycle_id`).

Le modèle permet à une étude d'utiliser plusieurs cycles et à un même cycle d'être associé à plusieurs études. Le Cycle ne possède donc pas de `study_id` et reste une entité indépendante et réutilisable.

Les clés étrangères de `study_cycles` utilisent ON DELETE CASCADE, mais le cascade porte uniquement sur la ligne d'association. Ainsi, la suppression d'une étude ne supprime pas les cycles, leurs résultats ou leurs plantes.

Cette architecture permet notamment la fonctionnalité « **Import Existing Cycle** », qui associe un cycle existant à une étude sans copier ses résultats ni ses informations sur les plantes.

Ainsi, l'association entre une étude et un cycle est entièrement portée par `study_cycles`, tandis que les données intrinsèques du cycle restent indépendantes de l'étude.

8.3 Théorie et implémentation de la gestion des fuseaux horaires

Fichier concerné : `backend/app/utils/timezone.py`.

8.3.1 Le principe : stockage UTC universel, conversion locale à la frontière

Règle d'or de toute application multi-fuseaux : stocker systématiquement les horodatages en UTC (temps universel, sans ambiguïté), et ne convertir vers/depuis un fuseau « métier » qu'aux points d'entrée/sortie humains (formulaire, fichier Excel, logs). C'est exactement la stratégie suivie ici : toutes les colonnes de date sont des `TIMESTAMP WITH TIME ZONE` PostgreSQL (toujours normalisées en UTC en interne par Postgres, quel que soit le fuseau de la session SQL).

8.3.2 Le fuseau métier configurable : `settings.TIMEZONE`

Une seule variable d'environnement (`TIMEZONE`, par défaut « Africa/Casablanca ») définit le fuseau dans lequel un humain « pense » une date. Elle est résolue UNE SEULE FOIS au chargement du module en un objet `ZoneInfo` nommé `APP_TZ` — toute autre partie du code doit importer `APP_TZ` (ou utiliser les fonctions ci-dessous) plutôt que de relire `settings.TIMEZONE` directement, pour garantir un seul point de vérité.

8.3.3 Les quatre fonctions de conversion

Fonction	Usage
<code>now_utc()</code>	Instant présent, aware, en UTC — pour tout <code>created_at/updated_at</code> stocké en base (remplace <code>datetime.utcnow()</code> , déprécié depuis Python 3.12).
<code>now_local()</code>	Instant présent, aware, dans le fuseau métier — pour les logs et toute logique raisonnant en « heure locale ».
<code>to_utc(value)</code>	Convertit une date SAISIE (formulaire, Excel) vers UTC pour stockage. Si la date est naïve (sans fuseau), elle est d'abord interprétée comme étant dans <code>APP_TZ</code> , puis convertie. Si elle est déjà aware, elle est simplement reconvertie en UTC sans réinterprétation — important pour ne jamais double-convertir un horodatage ThingSpeak déjà en UTC.
<code>to_local(value)</code>	Convertit une date LUE en base (toujours UTC) vers le fuseau métier, pour affichage/logs.

8.3.4 Piège connu : cohérence frontend/backend

Le frontend possède sa PROPRE variable `NEXT_PUBLIC_TIMEZONE` (`frontend/.env.local`), utilisée pour formater les dates affichées dans l'interface (`frontend/lib/format.ts`). Rien dans le code n'empêche cette valeur de diverger de la variable `TIMEZONE` du backend — il s'agit d'une convention à respecter manuellement, pas d'une contrainte vérifiée automatiquement. Si vous changez le fuseau horaire de l'application, modifiez IMPÉRATIVEMENT les deux variables ensemble.

8.4 Théorie de la machine à états des alertes

Fichier concerné : `backend/app/services/alert_service.py`.

8.4.1 Modèle conceptuel : compteur consécutif + seuil

Les deux détecteurs implémentés (No Data Collected, Capture Failure) suivent le même patron théorique, une machine à états à deux états (active / resolved) pilotée par un compteur d'échecs consécutifs comparé à un seuil N configurable :

- Tant que le compteur consécutif est strictement inférieur à N → aucune alerte n'existe (état implicite « ok », pas de ligne en base).
- Dès que le compteur atteint exactement N → une ligne `sensor_alerts` est créée, `status="active"`, `severity="warning"` → transition « ok » → « active », qui déclenche (une seule fois) les notifications externes.
- Si le compteur continue de croître au-delà de N → la même ligne est mise à jour (jamais dupliquée), `severity` passe à "critical" — pas de nouvelle notification à ce stade.
- Dès que la condition qui a déclenché l'alerte disparaît (nouvelle donnée reçue ; capture dont les valeurs ne sont plus toutes égales à 0) → `status` passe à "resolved", `resolved_at` est renseigné.

Ce patron évite deux pièges classiques : le spam de notifications (une notification n'est envoyée qu'à la transition « inactive → active », jamais à chaque évaluation périodique tant que l'alerte reste active) et la perte d'historique (une alerte résolue n'est jamais supprimée, seulement marquée comme telle, permettant une vue d'historique complète sur `/alerts`).

8.4.2 Où vit le compteur consécutif AVANT qu'une alerte n'existe ?

Point subtil souvent source de bugs dans ce genre de système : avant que le seuil N ne soit atteint, il n'existe encore AUCUNE ligne `sensor_alerts` sur laquelle stocker le compteur en cours. Le détecteur No Data Collected résout ce problème via une table dédiée, `collector_run_state` (une ligne par set de capteurs, avec `consecutive_empty_runs`), mise à jour à CHAQUE tentative de collecte, alerte ou non. Le détecteur Capture Failure, lui, n'a pas besoin d'une table d'état séparée : il recalcule le compteur à la volée en relisant les captures les plus récentes de `raw_sensor_data` (jusqu'à `LOOKBACK_LIMIT = 200` lignes) et en comptant celles qui sont « toutes à zéro » en partant de la plus récente, en s'arrêtant à la première capture valide (`_count_consecutive_capture_failures`).

8.4.3 Détection « both sets » vs détection individuelle par set

Le détecteur Capture Failure gère un cas particulier : si TOUS les sets configurés (`settings.valid_set_numbers`) échouent EN MÊME TEMPS, une seule alerte combinée est levée (`sensor_set="both"`) plutôt que N alertes individuelles redondantes — et toute alerte individuelle active pour un set devenu partie du groupe « both » est explicitement résolue (« superseded by All Sets ») pour éviter les doublons visuels sur le tableau de bord. Si un sous-ensemble seulement des sets échoue, chaque set en échec reste signalé individuellement. Cette logique est entièrement dans `detect_capture_failure()` et se généralise automatiquement si un troisième set de capteurs est ajouté (aucune valeur « 2 » n'est codée en dur : c'est `len(failing_sets) == len(set_numbers)`).

8.4.4 Pourquoi « tout à zéro » et pas « n'importe quel zéro » ?

`_is_capture_all_zero()` ne considère une capture en échec QUE si SES CINQ valeurs de paramètres (ph, temperature, ec, turbidity, do) sont simultanément égales à 0. Un pH de 0.0 isolé (une valeur individuellement légitime dans certains contextes, ou une erreur de calibration ponctuelle) ne déclenche jamais rien à lui seul — seule une ligne complètement dégénérée (typiquement le signe d'un capteur déconnecté ou d'un défaut ThingSpeak) compte comme un échec. C'est une heuristique volontairement conservatrice, documentée

comme limite connue (voir §6.4) : elle ne détecte pas une dérive progressive ou une valeur bloquée sur un nombre non nul.

8.5 Théorie de l'idempotence dans la collecte de capteurs

Fichier concerné : `backend/app/collectors/collector.py`.

8.5.1 Le problème : polling périodique et doublons

Un système qui interroge périodiquement une source externe (ici ThingSpeak) risque structurellement de récupérer plusieurs fois la même donnée — par exemple si deux exécutions consécutives se chevauchent sur les mêmes entrées récentes, ou si la collecte est relancée manuellement (`POST /api/collect`) juste après une collecte planifiée. Le patron d'ingénierie standard pour résoudre ce problème est l'IDEMPOTENCE : réexécuter la même opération plusieurs fois doit produire le même état final qu'une seule exécution.

8.5.2 Implémentation : contrainte unique + upsert PostgreSQL

Le projet garantit l'idempotence par construction, à deux niveaux complémentaires :

- Une contrainte `UNIQUE` en base de données sur (`entry_id`, `set_number`) — nommée `uq_raw_sensor_entry_set` — empêche PHYSIQUEMENT l'existence de deux lignes portant la même paire (`entry_id` ThingSpeak, numéro de set).
- Le code d'insertion utilise `INSERT ... ON CONFLICT DO NOTHING` (via `sqlalchemy.dialects.postgresql.insert().on_conflict_do_nothing()`) : si une ligne entrerait en collision avec cette contrainte, PostgreSQL l'ignore silencieusement plutôt que de lever une erreur — aucune logique applicative de type « vérifier si ça existe avant d'insérer » n'est nécessaire, ce qui élimine aussi toute race condition entre deux processus qui écriraient en parallèle.

`result.rowcount` après cet `INSERT` donne exactement le nombre de lignes RÉELLEMENT insérées (les lignes ignorées par `ON CONFLICT` ne comptent pas) — c'est cette valeur qui alimente ensuite le détecteur No Data Collected (voir §8.4.2).

8.5.3 Le backfill historique : une pagination inversée

`run_backfill()/fetch_all_feeds()` implémente une technique moins évidente : récupérer l'INTÉGRALITÉ de l'historique disponible sur un channel ThingSpeak (potentiellement des dizaines de milliers d'entrées), alors que l'API ThingSpeak ne renvoie qu'un nombre borné d'entrées par requête (`results`).

Piège découvert et documenté dans le code : lorsqu'on combine un paramètre `results` avec une borne de date `start/end` couvrant plus d'entrées que `results`, ThingSpeak ne renvoie PAS les `results` PREMIÈRES entrées de la plage, mais les `results` DERNIÈRES (les plus proches de `end`, ou de « maintenant » si `end` est omis). Une implémentation naïve qui avancerait `start` progressivement recevrait donc presque toujours les MÊMES entrées récentes en boucle, sans jamais progresser vers le passé.

La solution retenue est de paginer en reculant dans le temps : partir sans borne (ThingSpeak part alors de « maintenant »), noter le timestamp de l'entrée la plus ANCIENNE reçue dans la page, puis relancer la requête suivante avec `end` = ce timestamp. Les doublons de bordure entre deux pages consécutives sont filtrés via un ensemble `seen_entry_ids` en mémoire, puis de toute façon re-filtrés par la contrainte unique au moment du `store()`. Trois conditions d'arrêt indépendantes protègent contre une boucle infinie : une page vide, une page

strictement plus petite que `page_size` (signe que le début de l'historique est atteint), ou un plafond absolu `max_pages`.

Ce backfill s'exécute une seule fois, au démarrage du backend (voir `app/main.py::lifespan`, avant que le scheduler ne prenne le relais avec des collectes régulières bornées à `THINGSPEAK_RESULTS_COUNT` lignes).

8.6 Vocabulaire canonique et validation Pydantic

Fichier concerné : `backend/app/schemas/cycle.py`.

8.6.1 Le principe : un seul endroit de vérité pour le vocabulaire du domaine

`PARAMETERS` (les 8 paramètres), `PARAMETER_UNITS` (leurs unités), `STAGE_ROLES` (les 3 rôles d'étape stables) et `STAGE_LABELS` (leurs libellés d'affichage) sont définis UNE SEULE FOIS, dans `backend/app/schemas/cycle.py`. Toute validation d'un paramètre ou d'une étape ailleurs dans le code (côté import Excel, côté validation Pydantic des requêtes) importe ces constantes plutôt que de redéfinir sa propre liste — garantissant qu'ajouter un neuvième paramètre ne nécessite de modifier le code métier qu'à cet unique endroit (plus, séparément, `frontend/lib/constants.ts` qui doit rester synchronisé manuellement côté frontend).

8.6.2 Validation de complétude : les 24 lignes exactes

Le validateur Pydantic `rows_must_cover_every_parameter_stage_pair()` sur `CycleCreate` calcule l'ensemble théorique attendu (8 paramètres × 3 rôles = 24 paires) et le compare à l'ensemble effectivement reçu, rejetant la requête (erreur 422) si des paires manquent OU si des paires inattendues sont présentes. Cette validation est STRICTE : elle protège contre un payload partiel (bug frontend) ou mal formé, en échouant tôt et explicitement plutôt que de persister un cycle avec des trous silencieux.

8.6.3 Pourquoi utiliser Pydantic ici précisément

Pydantic (théorie : validation de données déclarative basée sur des annotations de type Python, avec levée d'erreurs structurées) permet de définir `CycleRowIn` comme un simple modèle Python typé, avec des `field_validator` supplémentaires pour les règles métier custom (`parameter_must_be_known`, `stage_must_be_known`) qui vont au-delà de ce qu'un simple typage permettrait. FastAPI utilise ensuite ces mêmes schémas pour générer automatiquement la documentation Swagger (`/docs`) et pour valider chaque requête HTTP entrante AVANT qu'elle n'atteigne la logique métier — aucune donnée malformée ne peut donc atteindre la couche services/.

8.7 Duplication contrôlée de la logique métier (frontend ↔ backend)

8.7.1 Pourquoi dupliquer plutôt que centraliser ?

Le frontend (frontend/lib/calculations.ts) réimplémente délibérément les mêmes formules que le backend (backend/app/services/calculations.py), en JavaScript/TypeScript plutôt que d'appeler l'API à chaque frappe clavier. C'est un compromis d'ingénierie assumé, pas un oubli : appeler le serveur à chaque caractère saisi dans le tableau de 24 lignes introduirait une latence réseau perceptible et une charge serveur inutile pour un simple aperçu visuel qui sera de toute façon RECALCULÉ ET REMPLACÉ par le backend au moment de l'enregistrement (POST /api/cycles).

8.7.2 Le contrat qui garantit la cohérence

Cette duplication n'est sûre que parce qu'un contrat implicite strict est maintenu entre les deux implémentations : mêmes formules mathématiques exactes (moyenne, écart-type ÉCHANTILLON avec n-1, formule de removal identique), même gestion des cas limites (valeurs manquantes → None/null, baseline à zéro → None/null jamais une erreur), et même arrondi (4 décimales des deux côtés). Le résultat final envoyé au serveur ne contient QUE les répliqués bruts (CycleRowIn ne transporte ni average, ni std, ni removal) — les valeurs calculées côté frontend ne sont donc JAMAIS elles-mêmes persistées : elles ne servent qu'à l'affichage temporaire pendant la saisie, et sont entièrement recalculées côté serveur avant stockage. Le backend reste ainsi la seule source de vérité pour les données réellement enregistrées.

Règle à respecter impérativement pour quiconque modifie une des deux formules : reporter le changement identique dans l'autre fichier, sinon l'aperçu affiché pendant la saisie divergera (même temporairement) de la valeur réellement enregistrée après clic sur « Save cycle ».

9. Guide pratique : comment modifier ou étendre le projet

9.1 Ajouter un neuvième paramètre de qualité de l'eau

- 1. backend/app/schemas/cycle.py : ajouter l'entrée dans PARAMETERS et son unité dans PARAMETER_UNITS.
- 2. frontend/lib/constants.ts : ajouter la même entrée côté frontend (liste séparée, à synchroniser manuellement — voir §8.6.1).
- 3. Aucune migration Alembic n'est nécessaire : cycle_results.parameter est une simple colonne String(64), pas une contrainte ENUM en base — la validation du vocabulaire se fait uniquement au niveau applicatif (Pydantic).
- 4. Vérifier que rows_must_cover_every_parameter_stage_pair() attend désormais 9×3=27 lignes — c'est automatique puisque la fonction dérive l'ensemble attendu de PARAMETERS dynamiquement.
- 5. Le formulaire frontend (composant CycleTable.tsx) doit afficher une ligne supplémentaire par étape — vérifier qu'il itère bien sur la constante partagée plutôt que sur une liste codée en dur de 8 éléments.

9.2 Ajouter un troisième (ou N-ième) set de capteurs ThingSpeak

- 1. Modifier UNIQUEMENT la variable d'environnement SENSOR_SETS (backend/.env), en ajoutant une entrée JSON {"set_number": 3, "channel_id": "...", "api_key": null}.
- 2. Aucune modification de code n'est nécessaire : settings.sensor_sets, settings.channel_map et settings.valid_set_numbers dérivent tous dynamiquement de cette configuration, et collect_all()/detect_capture_failure() itèrent sur ces listes sans jamais coder « 1 » ou « 2 » en dur.
- 3. Redémarrer le backend pour que la nouvelle configuration soit prise en compte (settings est chargé une seule fois au démarrage du processus).

9.3 Implémenter la détection d'anomalies (fonctionnalité placeholder)

- 1. backend/app/services/alert_service.py::detect_anomalies() a déjà la bonne signature (db, set_numbers) -> list[SensorAlert] mais retourne toujours [].
- 2. Définir la théorie du seuil : par exemple, un intervalle [min, max] valide par paramètre (pH entre 0 et 14, température entre 0°C et 100°C...), configurable idéalement via une nouvelle table ou une extension de alert_config.
- 3. Réutiliser le même patron que detect_capture_failure() : compteur consécutif par (set, paramètre), comparaison au seuil N existant (get_threshold), création/mise à jour via _upsert_active_alert() avec alert_type=AlertType.ANOMALY.value.
- 4. Le type AlertType.ANOMALY existe déjà dans backend/app/services/alert_types.py et dans le filtre frontend de la page /alerts — aucune modification de schéma n'est nécessaire côté affichage.

9.4 Ajouter les pages frontend manquantes (assign / export CSV)

- Les endpoints backend POST /api/study/{id}/assign et GET /api/study/{id}/export sont déjà pleinement implémentés côté backend, mais n'ont pas d'appel correspondant dans le client typé frontend/lib/api.ts : il manque une page ou un composant qui les appelle (et, pour l'export, la fonction cliente elle-même). C'est le point d'entrée le plus rapide pour un premier contributeur souhaitant se familiariser avec le pattern page → lib/api.ts → backend du projet.

9.5 Modifier le comportement d'import Excel

- Toute la logique vit dans `backend/app/services/cycle_import_service.py`. Les alias de colonnes acceptés (`_COLUMN_ALIASES`) et les libellés d'étape hérités (`_LEGACY_STAGE_LABEL_TO_ROLE`) sont des dictionnaires simples à étendre pour accepter de nouveaux noms de colonnes ou formats hérités, sans toucher au reste du pipeline de parsing.
- Le report d'erreurs par ligne (`row_errors`, voir `app/schemas/cycle_import.py`) fonctionne en accumulant les erreurs dans une liste puis en passant à la ligne suivante (`continue`) plutôt qu'en interrompant tout l'import — respecter ce patron pour toute nouvelle règle de validation ajoutée.

10. Référence API complète (paramètres, statuts, exemples)

Cette section détaille, endpoint par endpoint, ce que le code accepte et retourne réellement ;

10.1 Studies (/api/study)

Studies



POST	/api/study	Créer une étude	✓
GET	/api/study	Lister les études	✓
GET	/api/study/export	Exporter les mesures brutes d'une étude au format CSV	✓
GET	/api/study/{study_id}	Détail d'une étude	✓
DELETE	/api/study/{study_id}	Supprimer définitivement une étude	✓
POST	/api/study/{study_id}/assign	Assigner des mesures brutes existantes à une étude	✓
GET	/api/study/{study_id}/cycles	Lister les cycles de laboratoire d'une étude	✓
POST	/api/study/{study_id}/cycles/{cycle_id}/import	Importer/réutiliser un cycle existant (n'importe lequel) dans cette étude	✓
DELETE	/api/study/{study_id}/cycles/{cycle_id}	Retirer (unassign) un cycle de cette étude, sans le supprimer	✓
POST	/api/study/{study_id}/cycles/import	Importer des cycles depuis un fichier Excel (.xlsx / .xls)	✓

Endpoint	Détails
POST /api/study	Corps JSON : study_name, start_date, end_date (optionnel). Retourne 201 + StudyOut. Erreur 400 si end_date < start_date.
GET /api/study	Query params : skip (défaut 0), limit (défaut 100, max 1000). Pagination par offset simple.
GET /api/study/{id}/export	Filtre toujours appliqué : study_id (dans l'URL). Query params optionnels : start_date, end_date (ISO 8601), set_number — affinent l'export mais ne remplacent jamais le filtre par étude. Retourne un flux CSV (StreamingResponse) avec en-tête Content-Disposition en pièce jointe ; la colonne « time » du CSV est recalculée en minutes écoulées depuis la première ligne exportée.
GET /api/study/{id}	404 si l'étude n'existe pas.
POST /api/study/{id}/assign	Query param set_number obligatoire. Met à jour raw_sensor_data.study_id pour les lignes de ce set dans la plage de dates de l'étude, PAS ENCORE assignées. Ne copie ni ne duplique aucune donnée.

Endpoint	Détails
DELETE /api/study/{id}	204 si succès (aucun corps de réponse), 404 si l'étude n'existe pas. NE cascade PAS sur les cycles : seules les lignes study_cycles de l'étude sont supprimées (cycles/cycle_results/cycle_plants préservés) ; détache (SET NULL) raw_sensor_data et sensor_alerts.
GET /api/study/{id}/cycles	Retourne CycleSummaryOut (léger, sans les 24 lignes de résultats — pour l'affichage en liste).
POST /api/study/{study_id}/cycles/{cycle_id}/import	« Import Existing Cycle » : crée une ligne study_cycles reliant un cycle déjà existant (n'importe où en base) à cette étude. Erreur 409 si l'association existe déjà, 404 si l'étude ou le cycle n'existe pas. Ne copie ni ne clone aucune donnée du cycle.
DELETE /api/study/{study_id}/cycles/{cycle_id}	Retire (unassign) un cycle de cette étude en supprimant uniquement la ligne study_cycles correspondante — le cycle, ses résultats et ses noms de plantes restent intacts et disponibles pour toute autre étude. 404 si l'association n'existe pas.
POST /api/study/{id}/cycles/import	Multipart : champ file (.xlsx/.xls obligatoire), query param skip_duplicates (défaut true). Retourne CycleImportResult (total_rows, cycles_created, cycles_skipped, row_errors). Erreur 400 si l'extension du fichier n'est pas .xlsx/.xls.

10.2 Cycles (/api/cycles, /api/meta)

Cycles	
GET	/api/meta Paramètres, unités et rôles d'étape attendus pour un cycle
POST	/api/cycles Créer un cycle
GET	/api/cycles/{cycle_id} Détail d'un cycle
DELETE	/api/cycles/{cycle_id} Supprimer définitivement un cycle

Endpoint	Détails
GET /api/meta	Retourne PARAMETERS, PARAMETER_UNITS, STAGE_ROLES, STAGE_LABELS et BASELINE_STAGE_ROLE — le vocabulaire canonique, pour que le frontend construise dynamiquement ses formulaires/en-têtes sans dupliquer ces listes.
POST /api/cycles	Corps : CycleCreate (study_id, cycle_name, start_date, end_date, plants, rows[24]). Validation stricte des 24 lignes (voir §8.6.2). Crée le cycle, ses 24 cycle_results ET la première ligne study_cycles reliant le cycle à l'étude, en une seule transaction. Retourne le cycle créé avec ses résultats calculés.
GET /api/cycles/{id}	Retourne CycleOut complet (résultats + plants), via un eager-load joinedload (results, plants) pour éviter les requêtes N+1.

Endpoint	Détails
GET /api/cycles/available-for-import	Liste TOUS les cycles de la base, sans filtre par étude, chacun annoté des études qui l'utilisent déjà ; paramètre optionnel search (insensible à la casse sur cycle_name). Alimente la fenêtre « Import Existing Cycle » du frontend.

10.3 Alerts (/api/alerts)

Alerts ^	
GET	/api/alerts Lister les alertes
GET	/api/alerts/config Consulter le seuil d'alerte
PUT	/api/alerts/config Modifier le seuil d'alerte

Endpoint	Détails
GET /api/alerts	Filtres optionnels en query params : alert_type, study_id, sensor_set, severity, status, date_from, date_to, skip, limit (défaut 200).
GET /api/alerts/config	Retourne le seuil N actuellement configuré (alert_config.null_capture_threshold, ou la valeur par défaut des settings si aucune ligne n'existe encore).
PUT /api/alerts/config	Corps : { "null_capture_threshold": N }. Crée ou met à jour la ligne singleton id=1 de alert_config.

10.4 Raw data & Health

Raw Sensor Data ^	
GET	/api/raw-data Lister les données brutes
GET	/api/raw-data/latest Dernière mesure disponible
GET	/api/raw-data/set/{set_number} Données d'un set de capteurs
Health ^	
GET	/ Healthcheck
GET	/api/sensor-sets Sets de capteurs configurés
POST	/api/collect Déclencher manuellement une collecte ThingSpeak
default ^	
GET	/api/scheduler Scheduler Status

Endpoint	Détails
GET /	Healthcheck simple : {status, app, version}.
GET /api/sensor-sets	Liste les sets configurés dynamiquement (set_number, channel_id), pour que le frontend n'ait jamais à coder « Set 1 »/« Set 2 » en dur.
POST /api/collect	Déclenche immédiatement collect_all() en dehors de la planification APScheduler — utile pour tester ou forcer une collecte.

Endpoint	Détails
GET /api/scheduler	Retourne l'état du scheduler APScheduler (running, liste des jobs avec next_run_time).
GET /api/raw-data	Liste paginée des mesures brutes, avec filtres (voir app/routes/raw_data.py pour la liste exacte des query params).
GET /api/raw-data/latest	Dernière mesure disponible, tous sets confondus ou filtrée par set.
GET /api/raw-data/set/{set_number}	Mesures brutes d'un set de capteurs spécifique.

11. Variables d'environnement et configuration — référence complète

Fichier de référence : backend/app/config.py (classe Settings, chargée depuis backend/.env via pydantic-settings).

Variable	Défaut	Rôle
APP_NAME	Water Quality Monitoring API	Titre affiché dans la documentation Swagger.
LOG_LEVEL	INFO	Niveau de verbosité des logs applicatifs.
TIMEZONE	Africa/Casablanca	Fuseau métier unique — voir §8.3. Doit être un nom de zone IANA valide.
DATABASE_URL	postgresql://...	Chaîne de connexion SQLAlchemy vers PostgreSQL.
CORS_ORIGINS	localhost:3000, 127.0.0.1:3000	Liste JSON des origines autorisées à appeler l'API depuis un navigateur.
THINGSPEAK_BASE_URL	https://api.thingspeak.com	Base de l'API ThingSpeak (modifiable pour un proxy ou un environnement de test).
SENSOR_SETS	JSON (set 1 + set 2)	Liste JSON des channels ThingSpeak configurés — voir §9.2 pour en ajouter un.
COLLECTOR_INTERVAL_MINUTES	5	Fréquence de la collecte planifiée (APScheduler).
THINGSPEAK_RESULTS_COUNT	10	Nombre d'entrées récupérées par collecte régulière (hors backfill initial, qui récupère tout l'historique).
NULL_CAPTURE_THRESHOLD	3	Seuil N par défaut avant qu'une alerte ne devienne active — surchargeable à chaud via PUT /api/alerts/config.
SMTP_HOST / SMTP_PORT / SMTP_USERNAME / SMTP_PASSWORD / SMTP_FROM_EMAIL / SMTP_USE_TLS	vide / 587 / — / — / — / true	Configuration email des alertes ; incomplète → notifications email silencieusement désactivées.
ALERT_EMAIL_RECIPIENTS	vide	Liste d'e-mails séparés par des virgules, destinataires des alertes.
TELEGRAM_BOT_TOKEN / TELEGRAM_CHAT_ID	vide / vide	Configuration Telegram ; l'un des deux manquant → notifications Telegram silencieusement désactivées.

Côté frontend, frontend/.env.local (copié depuis .env.local.example) définit typiquement

NEXT_PUBLIC_API_URL (URL du backend) et NEXT_PUBLIC_TIMEZONE (voir §8.3.4 pour l'importance de la cohérence avec le backend).

11.1 Démarrage pas à pas (rappel condensé)

- Backend : créer un environnement virtuel Python, pip install -r requirements.txt, copier .env.example vers .env et l'éditer, créer la base PostgreSQL, alembic upgrade head, puis uvicorn app.main:app --reload.
- Frontend : npm install, copier .env.local.example vers .env.local, puis npm run dev.
- Ordre recommandé : démarrer le backend en premier (le frontend échoue silencieusement sur ses appels fetch si le backend n'est pas joignable — voir §6.3, gestion des erreurs frontend).

12. Glossaire des termes techniques et métier

Terme	Définition dans le contexte de ce projet
Étude (Study)	Regroupement de haut niveau d'une campagne expérimentale : ses dates, et tous les cycles/mesures qui lui sont rattachés.
Cycle	Une campagne de laboratoire ponctuelle (saisie manuelle ou import Excel) : 24 lignes de résultats (8 paramètres × 3 étapes) plus jusqu'à 3 noms de plantes, rattachée à une étude.
study_cycles (réutilisation de cycle)	Table d'association many-to-many entre Study et Cycle, introduite par la migration 0003. Permet à un même cycle d'être rattaché à plusieurs études sans duplication, via le bouton « Import Existing Cycle » du frontend — voir §4.2.
Réplicat (Replicate)	Une des 3 mesures répétées d'un même (paramètre, étape) dans un cycle, utilisée pour calculer moyenne et écart-type.
Baseline (stage_1 / Wastewater)	L'échantillon brut/non traité, utilisé comme référence pour calculer le pourcentage d'abattement des autres étapes.
Removal % (abattement)	Pourcentage de réduction d'un paramètre entre la baseline et un échantillon traité — voir formule §8.1.3.
Écart-type d'échantillon (Sample std, ddof=1)	Mesure de dispersion statistique utilisant n-1 au dénominateur (par opposition à l'écart-type de population, ddof=0) — la même convention qu'Excel STDEV.
Sensor set	Un channel ThingSpeak configuré (typiquement « Set 1 », « Set 2 »), correspondant à un dispositif physique de capteurs.
Idempotence	Propriété d'une opération dont l'exécution répétée produit le même résultat qu'une seule exécution — ici garantie par la contrainte unique (entry_id, set_number) + ON CONFLICT DO NOTHING.
Upsert	Opération combinant insertion et mise à jour conditionnelle (« insert or update ») — ici utilisée sous sa forme « insert or ignore » via ON CONFLICT DO NOTHING.
Backfill	Récupération rétroactive de l'historique complet d'une source de données, par opposition à la collecte incrémentale régulière.
Alerte active / résolue	Les deux états du cycle de vie d'une alerte — voir machine à états détaillée en §8.4.
CORS (Cross-Origin Resource Sharing)	Mécanisme du navigateur limitant les requêtes HTTP entre origines différentes ; ici configuré via une liste blanche (CORS_ORIGINS), pas un mécanisme d'authentification.
Migration (Alembic)	Script versionné permettant de créer ou de faire évoluer le schéma de la base de données. Dans ce projet, le schéma final est créé par une migration Alembic consolidée : 0001_initial_consolidated.py. Elle regroupe directement la création des tables, relations, contraintes et index nécessaires au système.
ORM (Object-Relational Mapping)	Technique consistant à représenter des tables SQL sous forme de classes Python (ici SQLAlchemy) pour manipuler la base sans écrire de SQL brut.
Pydantic	Bibliothèque Python de validation de données basée sur des annotations de type, utilisée ici pour valider chaque requête/réponse de l'API.

Chapitre II — Conception et réalisation du système d'authentification et de gestion des utilisateurs

II.1 Introduction

La sécurisation de l'accès aux applications et la gestion différenciée des utilisateurs constituent aujourd'hui un socle indispensable de toute plateforme logicielle exposée sur le réseau. Dans le cadre du projet, il a été nécessaire de concevoir et de développer un module autonome d'authentification et de gestion des utilisateurs, destiné à contrôler l'accès à l'application et à distinguer les niveaux de privilèges accordés à chaque catégorie d'utilisateurs (administrateur, utilisateur standard, invité).

Ce chapitre présente en détail la conception, l'implémentation et la validation de ce module. Il couvre l'ensemble de la chaîne technique : l'architecture générale (frontend React, backend Node.js/Express, base de données PostgreSQL, service de messagerie SMTP), les mécanismes de sécurité mis en œuvre (hachage des mots de passe, jetons JWT, limitation de débit, validation des entrées), ainsi que les flux fonctionnels spécifiques développés pour ce module : inscription, connexion, récupération de mot de passe et approbation des comptes administrateurs par courrier électronique.

L'ensemble des éléments décrits dans ce chapitre correspond strictement à l'implémentation réelle du module tel qu'il a été développé ; aucune fonctionnalité, technologie ou donnée de test n'a été ajoutée artificiellement à des fins de présentation.

II.2 Présentation générale du module

Le module développé se présente comme un sous-système autonome et réutilisable, découplé du reste de l'application métier, chargé exclusivement de l'authentification et de la gestion des comptes utilisateurs. Il repose sur une architecture client-serveur classique à trois niveaux : une interface utilisateur React (Vite) côté client, une API REST Node.js/Express côté serveur, et une base de données relationnelle PostgreSQL pour la persistance des données.

Le module a été intégré dans le cadre du projet plus large de supervision de la qualité de l'eau (« Smart Water Quality Monitoring »), dont il constitue la brique de contrôle d'accès. Il expose une API préfixée par /api, consommée aussi bien par le frontend applicatif que, ponctuellement, par des liens d'action envoyés par courrier électronique (approbation/rejet de comptes administrateurs).

Principales fonctionnalités livrées

- Inscription et connexion avec validation stricte côté serveur (express-validator)
- Authentification par jetons JWT (access token / refresh token) transmis via cookies httpOnly
- Gestion des rôles admin / user / guest, avec contrôle d'accès au niveau des routes et des contrôleurs
- Flux « mot de passe oublié / réinitialisation » avec jetons à usage unique envoyés par e-mail
- Flux d'approbation des comptes administrateurs auto-inscrits, avec approbation ou rejet en un clic depuis un e-mail
- Limitation de débit (rate limiting) sur l'ensemble des routes sensibles
- Interface d'administration pour la recherche, la modification, l'activation/désactivation et la suppression des comptes
- Schéma de base de données versionné via des migrations SQL successives

II.3 Analyse des besoins

II.3.1 Acteurs du système

Technologie	Utilisation
React	Interface utilisateur
Vite	Build frontend
Node.js	Runtime backend
Express.js	API REST
PostgreSQL	Base de données
JWT	Authentification
bcrypt	Hachage
Nodemailer	Email
Axios	Communication HTTP
React Router	Navigation
express-validator	Validation
express-rate-limit	Protection contre les abus

II.3.2 Besoins fonctionnels

- Permettre à un visiteur de créer un compte (nom complet, nom d'utilisateur, e-mail, mot de passe)
- Permettre à un utilisateur de s'authentifier via son e-mail ou son nom d'utilisateur
- Empêcher la création de comptes en double (e-mail ou nom d'utilisateur déjà utilisés)
- Restreindre l'attribution directe du rôle administrateur : toute demande de compte admin doit passer par une validation humaine
- Permettre à un utilisateur de demander la réinitialisation de son mot de passe via un lien envoyé par e-mail
- Permettre à un administrateur de consulter, rechercher, paginer, modifier et supprimer les comptes existants
- Permettre à un administrateur d'approuver ou de rejeter une demande de compte administrateur, depuis le tableau de bord ou directement depuis un e-mail
- Garantir la déconnexion effective de l'utilisateur (invalidation des cookies de session)

II.3.3 Besoins non fonctionnels

- Sécurité : hachage des mots de passe, jetons signés et à durée de vie limitée, protection contre l'énumération des comptes, limitation du débit sur les routes sensibles
- Fiabilité : gestion explicite des erreurs, réponses HTTP cohérentes, transactions atomiques pour les opérations critiques (réinitialisation de mot de passe, approbation de compte)
- Maintenabilité : séparation claire des couches (routes, contrôleurs, services, accès aux données), schéma de base de données versionné par migrations
- Portabilité/configuration : paramétrage exclusivement par variables d'environnement (aucun identifiant ni secret en dur dans le code)
- Ergonomie : formulaires avec validation en temps réel côté client, messages d'erreur explicites, retours visuels de chargement

II.4 Conception de l'architecture

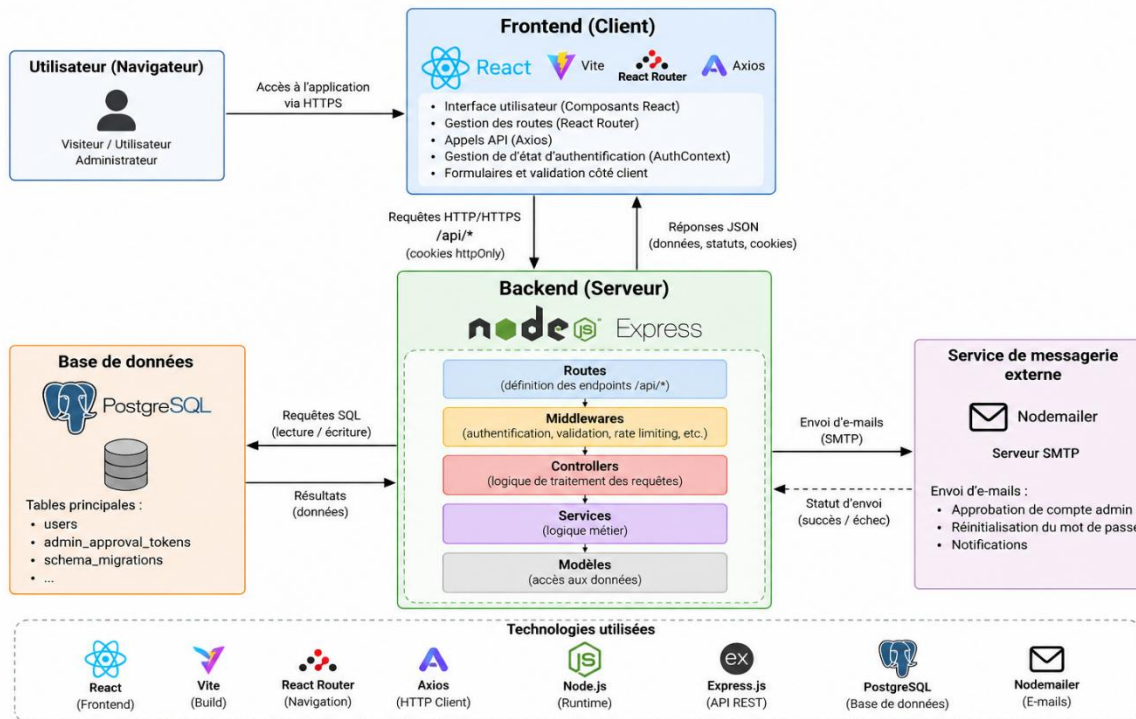


Figure II.1 — Architecture générale du système d'authentification et de gestion des utilisateurs

L'architecture globale du module suit un modèle client-serveur en couches. Le frontend React communique avec l'API Express exclusivement via des requêtes HTTP (Axios), en s'appuyant sur des cookies httpOnly pour le transport des jetons de session. Le backend s'articule autour de cinq couches logicielles distinctes : les routes (définition des points d'entrée et des règles de validation/limitation), les contrôleurs (logique de traitement des requêtes), les services (logique métier réutilisable : hachage, signature de jetons, envoi d'e-mails), le modèle d'accès aux données (requêtes SQL paramétrées vers PostgreSQL) et les middlewares transverses (authentification, autorisation, validation).

II.4.1 Style architectural

Le backend expose une API REST stateless du point de vue applicatif : chaque requête contient les informations nécessaires à son traitement (jeton d'accès transmis par cookie ou en-tête Authorization), sans session serveur conservée en mémoire. L'état d'authentification est entièrement porté par les jetons JWT signés, ce qui permet au backend de rester horizontalement scalable.

II.4.2 Organisation des dossiers du projet

```

backend/
├── config/           # Connexion à la base de données (pool pg)
├── controllers/      # authController, userController, adminController,
                        adminRegistrationController
├── middleware/       # authMiddleware (requireAuth / requireRole),
                        validateRequest
├── migrations/       # Migrations SQL versionnées + script d'exécution (migrate.js)
├── models/           # userModel.js – accès aux données (requêtes SQL paramétrées)
├── routes/           # authRoutes, userRoutes, adminRegistrationRoutes
├── services/         # authService (JWT, hachage, jetons), emailService (Nodemailer)
├── smtp_diagnose.js  # Script autonome de diagnostic de connexion SMTP
└── server.js         # Point d'entrée de l'application Express

frontend/
├── src/
│   └── components/   # Formulaires (Login, Register, ForgotPassword, ResetPassword),

```

	Navbar, PasswordInput, AuthIllustration
— context/	# AuthContext (état global de session)
— pages/	# Pages de routage : LoginPage, RegisterPage, DashboardRouter, AdminDashboard, UserDashboard, GuestDashboard, AdminUsersPage, UserProfile, Unauthorized, NotFound
— routes/	# ProtectedRoute, RoleProtectedRoute
— services/	# api.js (client Axios), authService.js, userService.js

II.5 Implémentation du Backend

Le backend repose sur Node.js et le framework Express. Le point d'entrée `server.js` configure les middlewares globaux (CORS restreint à l'origine du frontend avec credentials activés, parsing JSON, parsing des cookies), déclare les trois routeurs applicatifs (`/api/auth`, `/api/users`, `/api/admin-registration`), expose un point de contrôle de santé (GET `/api/health`) et met en place un gestionnaire d'erreurs global ainsi qu'un gestionnaire 404 pour les routes `/api` non reconnues.

II.5.1 Couche routes

Chaque route déclare explicitement ses règles de validation (via `express-validator`) et, le cas échéant, son middleware de limitation de débit, avant d'invoquer le contrôleur correspondant. Cette approche centralise la validation en amont du contrôleur et évite la duplication de vérifications dans la logique métier.

II.5.2 Couche contrôleurs

Les contrôleurs (`authController`, `userController`, `adminController`, `adminRegistrationController`) orchestrent le traitement des requêtes : lecture des paramètres, appel au modèle de données et aux services, construction des réponses HTTP. Ils ne contiennent aucune requête SQL directe : tout accès à la base transite par le modèle `userModel`.

II.5.3 Couche services

Le service `authService.js` centralise la logique cryptographique : hachage/vérification des mots de passe (bcrypt, 12 rounds de sel), signature et vérification des jetons JWT (access et refresh), ainsi que la génération de jetons aléatoires sécurisés pour la réinitialisation de mot de passe et l'approbation des comptes administrateurs (32 octets aléatoires via le module `crypto` de Node.js, hachés en SHA-256 avant stockage).

Le service `emailService.js` encapsule l'envoi de courriers électroniques via Nodemailer : construction d'un transporteur SMTP unique et partagé (paramétré uniquement par variables d'environnement), génération de gabarits HTML/texte pour chacun des quatre types d'e-mails envoyés par le système (réinitialisation de mot de passe, demande d'approbation administrateur, approbation confirmée, rejet confirmé), ainsi que la génération de pages HTML de résultat affichées dans le navigateur après un clic sur un lien d'approbation/rejet.

II.5.4 Couche modèle de données

Le module `userModel.js` constitue l'unique point d'accès à la table `users` et à la table `admin_approval_tokens`. Toutes les requêtes utilisent des paramètres préparés (`$1`, `$2...`) plutôt que la concaténation de chaînes, ce qui élimine par construction le risque d'injection SQL. Les opérations sensibles impliquant plusieurs écritures liées (réinitialisation de mot de passe, approbation/rejet d'un compte administrateur via jeton) sont exécutées dans des transactions PostgreSQL explicites (BEGIN/COMMIT/ROLLBACK), avec verrouillage de ligne (SELECT ... FOR UPDATE) pour les opérations d'approbation par jeton, afin d'empêcher toute double utilisation d'un même jeton en cas de requêtes concurrentes.

II.6 Conception et gestion de la base de données

La base de données utilisée est PostgreSQL. Le schéma est géré exclusivement par un ensemble de migrations SQL versionnées, appliquées dans l'ordre alphabétique par un script d'exécution (`migrations/migrate.js`) qui journalise chaque migration appliquée dans une table technique `schema_migrations`, garantissant l'idempotence des exécutions successives.

6.1 Structure de la table users

La table `admin_approval_tokens` associe à chaque demande de compte administrateur un jeton (dont seul le hachage SHA-256 est stocké), sa date d'expiration (24 heures) et sa date d'utilisation éventuelle, avec une clé étrangère vers `users` (ON DELETE CASCADE).

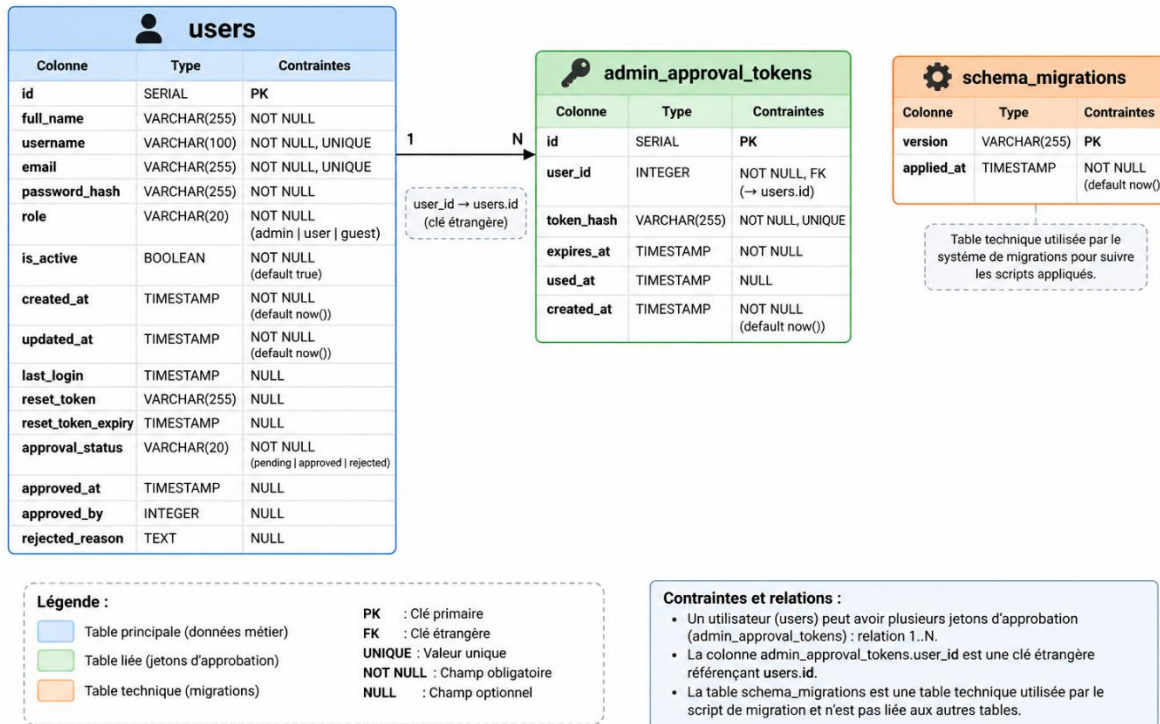


Figure II.2 — Modèle de données du système d'authentification et de gestion des

II.7 Développement du Frontend

Le frontend est développé avec React 18 et le bundler Vite, en JavaScript (fichiers .jsx). La navigation est gérée par React Router v6, et les appels HTTP vers l'API sont centralisés dans un client Axios unique (`services/api.js`), configuré avec `withCredentials: true` afin de transmettre automatiquement les cookies `httpOnly` de session à chaque requête.

7.1 Gestion de l'état d'authentification

Le contexte `AuthContext` (React Context API) centralise l'état de session côté client : utilisateur courant, indicateur de chargement initial, ainsi que les actions `login`, `register`, `logout` et `refreshUser`. Au montage de l'application, il tente automatiquement de récupérer l'utilisateur courant via `GET /api/auth/me` pour restaurer une session existante à partir du cookie.

7.2 Routage protégé

Deux composants de garde encadrent les routes sensibles : `ProtectedRoute` redirige vers `/login` tout visiteur non authentifié ; `RoleProtectedRoute` restreint l'accès à un sous-ensemble de rôles autorisés (utilisé pour `/admin/users`, limité au rôle `admin`) et redirige vers une page `/unauthorized` dédiée en cas d'accès refusé. Le code source précise explicitement que cette protection côté client constitue un confort d'expérience utilisateur, et que le contrôle d'autorisation réel est systématiquement revérifié côté serveur.

7.3 Composants et pages principaux

- `LoginForm`, `RegisterForm`, `ForgotPasswordForm`, `ResetPasswordForm` : formulaires avec validation côté client et gestion des erreurs serveur
- `PasswordInput` : champ de saisie de mot de passe avec bascule d'affichage/masquage

- Navbar : barre de navigation adaptative selon l'état de connexion et le rôle de l'utilisateur
- DashboardRouter : redirige vers AdminDashboard, UserDashboard ou GuestDashboard selon le rôle du compte connecté
- AdminUsersPage : interface d'administration (recherche, pagination, changement de rôle, activation/désactivation, suppression avec confirmation)
- AdminDashboard intègre le panneau « Pending Admin Requests », listant les demandes de compte administrateur en attente et permettant leur approbation/rejet
- UserProfile : consultation et modification des informations personnelles de l'utilisateur connecté

CIRC UA

Log in to your Account

Welcome back! Select a method to log in:

Email

you@example.com

Password

Remember me

Forgot Password?

Log in

Don't have an account? [Create an account](#)

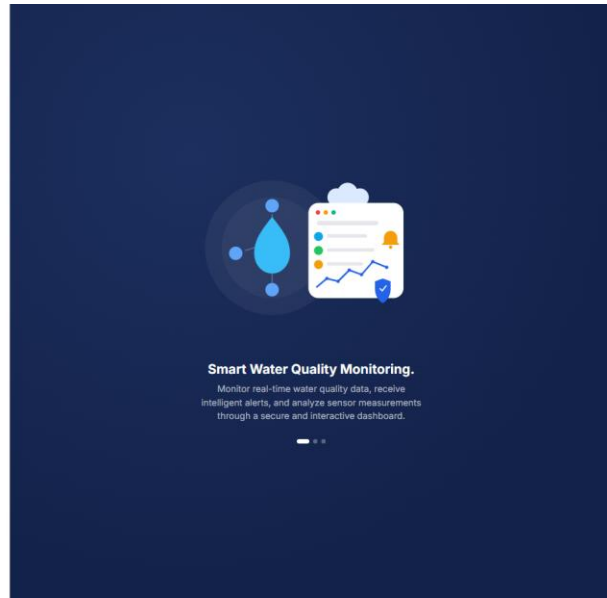


Figure II.3 — capture d'écran de la page de connexion (mise en page « split-screen » avec formulaire à gauche et illustration à droite).

CIRC UA

Create your Account

Standard access by default — admin roles are granted separately.

Full name

Jane Doe

Username

janedoe

Email

you@example.com

Password

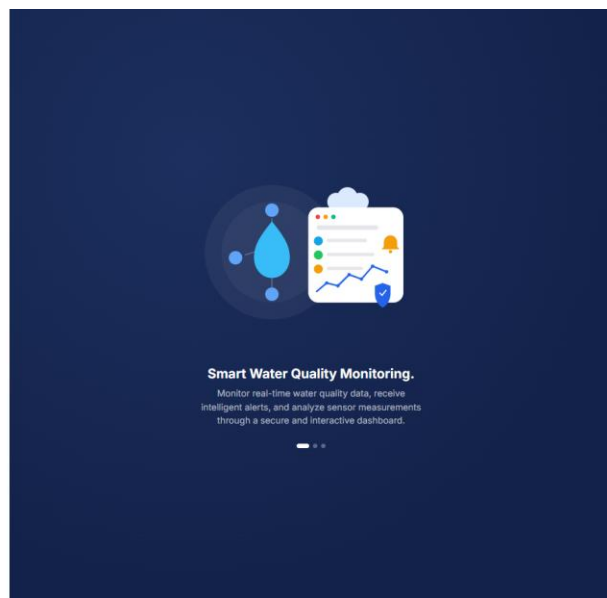
Confirm Password

Account type

User

Create account

Already have an account? [Log in](#)



capture d'écran de la page de Register (mise en page « split-screen » avec formulaire à gauche et illustration à droite).

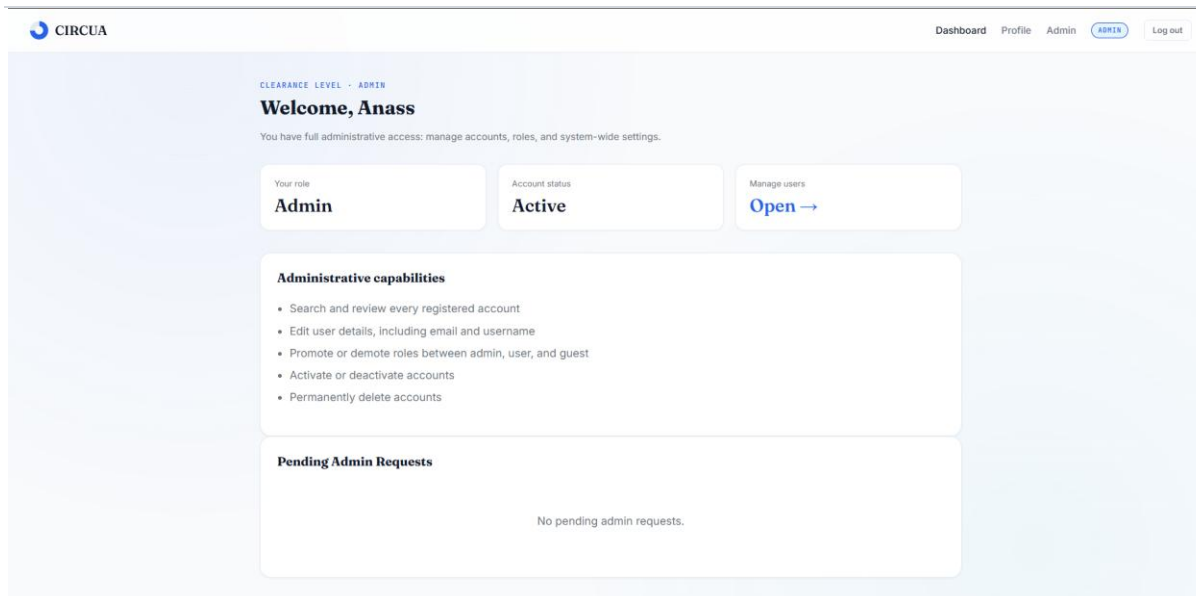


Figure II.4 — capture d'écran du tableau de bord administrateur, incluant le panneau des demandes d'administrateur en attente et le lien vers la gestion des utilisateurs.

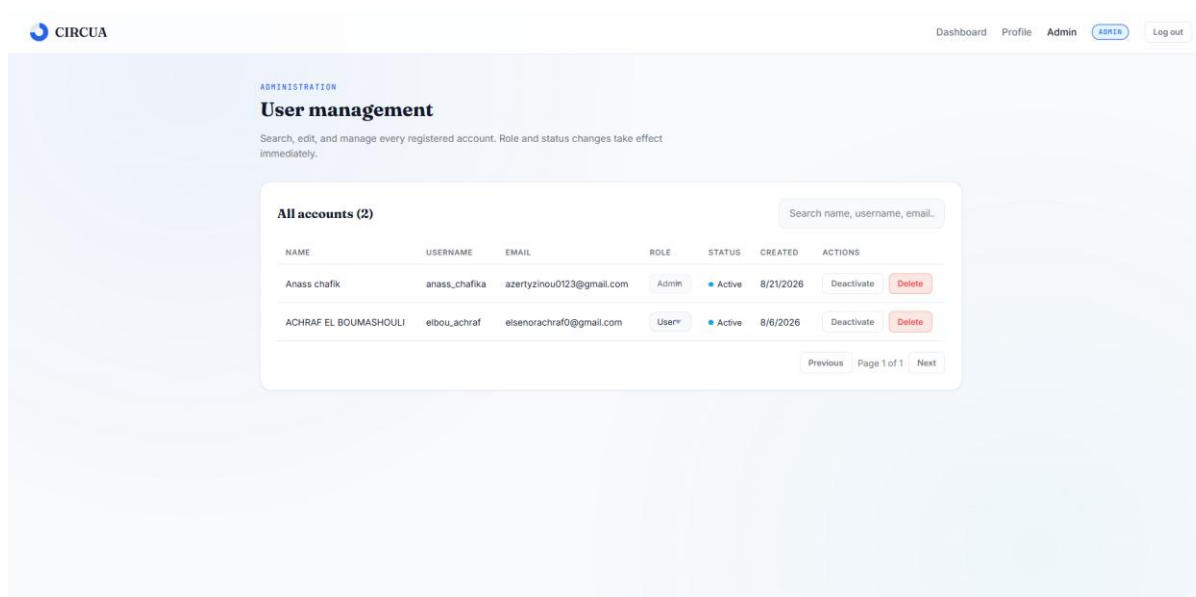


Figure II.5 —capture d'écran de la page de gestion des utilisateurs (liste paginée, recherche, changement de rôle, activation/désactivation, suppression).

II.8 Fonctionnalités de sécurité

8.1 Hachage des mots de passe

Les mots de passe ne sont jamais stockés en clair. Ils sont hachés avec l'algorithme bcrypt (12 rounds de sel), ce qui rend impraticable toute attaque par force brute ou par table arc-en-ciel en cas de fuite de la base de données.

8.2 Authentification par JWT et cookies httpOnly

À la connexion, le serveur émet un jeton d'accès (durée de vie configurable, 15 minutes par défaut) et un jeton de rafraîchissement (7 jours par défaut), tous deux signés avec des secrets distincts (JWT_SECRET et JWT_REFRESH_SECRET). Ces jetons sont déposés dans des cookies httpOnly (inaccessibles en JavaScript côté client, ce qui limite l'exposition aux attaques XSS), avec l'attribut sameSite=lax et l'attribut secure activé en production (COOKIE_SECURE=true). Le jeton d'accès est également renvoyé dans le corps de la réponse JSON pour les clients préférant l'en-tête Authorization: Bearer.

8.3 Contrôle d'accès basé sur les rôles (RBAC)

Le middleware requireAuth vérifie la validité du jeton d'accès puis recharge systématiquement l'utilisateur depuis la base de données (et non depuis les seules informations encodées dans le jeton), afin de garantir que le rôle et le statut actif utilisés pour l'autorisation reflètent toujours l'état courant du compte. Le middleware requireRole(...roles) restreint ensuite l'accès à certaines routes aux rôles autorisés. Cette double vérification (existence + fraîcheur des données) empêche qu'un jeton resté valide continue de donner accès à un compte désactivé ou rétrogradé entre-temps.

8.4 Jetons à usage unique pour les opérations sensibles

Les jetons de réinitialisation de mot de passe et les jetons d'approbation/rejet de comptes administrateurs partagent le même modèle de sécurité : générés aléatoirement (32 octets, module crypto de Node.js), transmis en clair uniquement dans l'e-mail envoyé à l'utilisateur, et stockés côté serveur sous forme de hachage SHA-256 uniquement. Ainsi, une fuite de la base de données ne permet pas, à elle seule, de reconstituer un jeton valide. Chaque jeton est à usage unique et expire automatiquement (15 minutes pour la réinitialisation de mot de passe, 24 heures pour l'approbation administrateur).

8.5 Limitation de débit (rate limiting)

Le module express-rate-limit est appliqué sur l'ensemble des routes sensibles à des seuils différenciés : connexion (10 tentatives / 15 minutes), inscription (20 / heure), mot de passe oublié (5 / 15 minutes), réinitialisation de mot de passe (10 / 15 minutes) et liens d'approbation/rejet d'administrateur (30 / 15 minutes). Cette mesure limite les attaques par force brute ainsi que les abus (bombardement d'e-mails, sondage systématique des jetons).

8.6 Prévention de l'énumération des comptes

Le point d'entrée /api/auth/forgot-password renvoie systématiquement le même message générique, que l'adresse e-mail corresponde ou non à un compte existant, et ce même en cas d'erreur interne inattendue. Ce comportement délibéré empêche un attaquant d'utiliser cet endpoint pour déterminer quelles adresses e-mail sont enregistrées dans le système.

8.7 Séparation des privilèges pour la modification des comptes

Le contrôleur userController distingue explicitement les champs qu'un utilisateur peut modifier sur son propre profil (nom, nom d'utilisateur, e-mail) des champs réservés aux administrateurs (rôle, statut actif). Toute tentative de modification du rôle ou du statut par un utilisateur non-administrateur est rejetée avec un code HTTP 403, quelle que soit la valeur envoyée par le client — le rôle transmis par le frontend n'est en aucun cas considéré comme une source de vérité pour les décisions d'autorisation.

8.8 Protection contre l'injection SQL et l'injection HTML

Toutes les requêtes SQL du modèle userModel utilisent des paramètres préparés, éliminant le risque d'injection SQL. Les champs utilisateur intégrés dans les gabarits HTML des e-mails (nom complet, nom d'utilisateur, e-mail) sont systématiquement échappés (fonction escapeHtml) avant insertion dans le corps HTML, afin d'empêcher qu'un utilisateur malveillant n'injecte du contenu HTML ou un script dans un e-mail destiné à un administrateur.

8.9 Configuration par variables d'environnement

Aucun identifiant, secret ou clé cryptographique n'est codé en dur dans le code source : les secrets JWT, les identifiants de connexion à PostgreSQL et les identifiants SMTP sont exclusivement lus depuis les variables d'environnement (fichier .env, non versionné), avec un fichier .env.example fourni comme gabarit documenté.

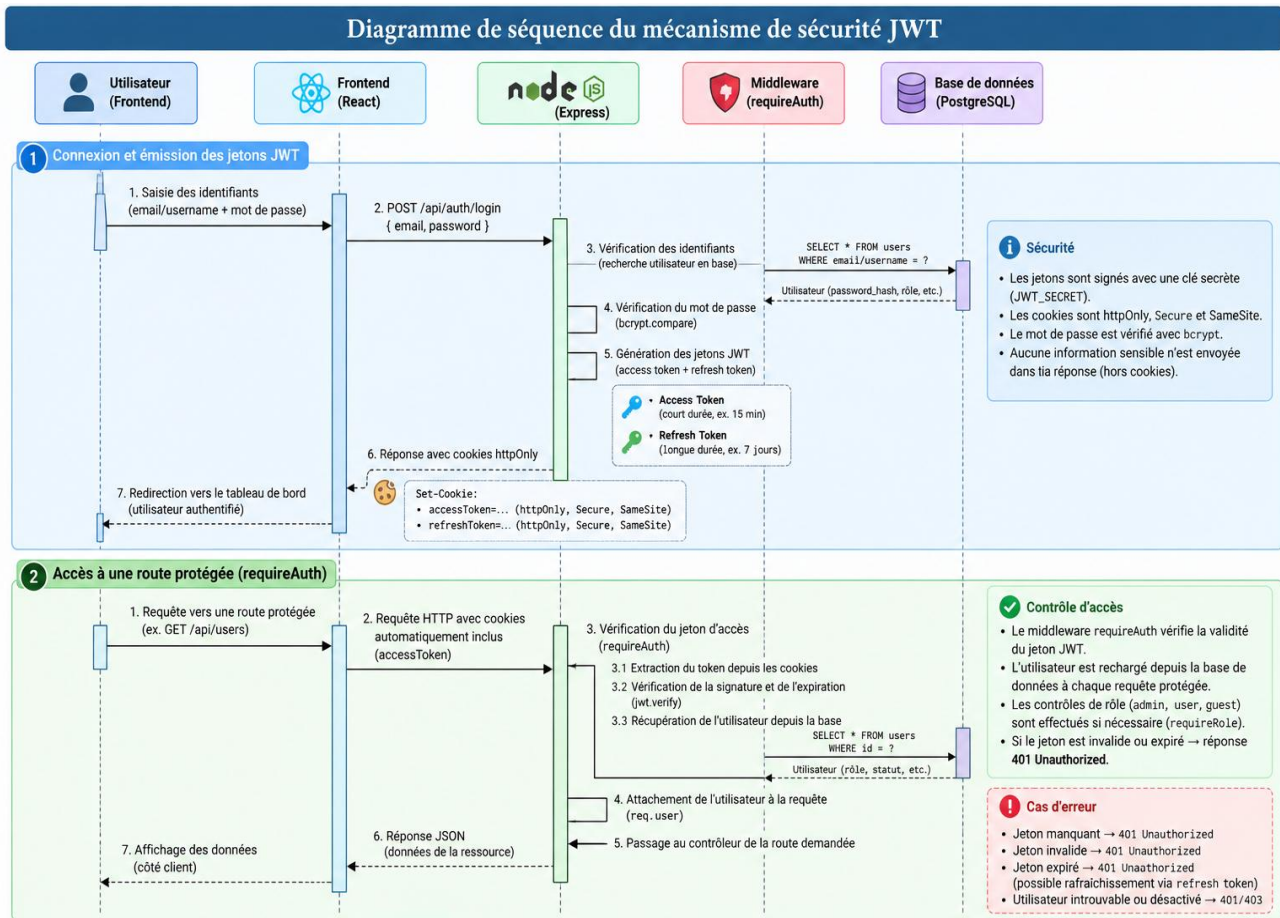


Figure II.6 — diagramme de séquence détaillé du mécanisme de sécurité JWT

II.9 Processus d'inscription et de connexion

9.1 Inscription (POST /api/auth/register)

Le formulaire d'inscription collecte le nom complet, le nom d'utilisateur, l'adresse e-mail, le mot de passe (avec confirmation) et un rôle souhaité (user par défaut, guest ou admin). Côté serveur, express-validator applique des règles strictes : nom complet non vide (2 à 150 caractères), nom d'utilisateur de 3 à 50 caractères limité aux lettres, chiffres et aux caractères « _ . - », e-mail syntaxiquement valide et normalisé, mot de passe d'au moins 8 caractères contenant au moins une majuscule et un chiffre, et concordance entre le mot de passe et sa confirmation.

Le contrôleur vérifie ensuite l'absence de doublon sur l'e-mail et le nom d'utilisateur avant toute création. Si le rôle demandé est admin, le compte est créé avec un statut approval_status = 'pending' et immédiatement désactivé (is_active = false) : il ne peut donc pas se connecter tant qu'il n'a pas été approuvé. Un jeton d'approbation à usage unique est alors généré et un e-mail de notification est envoyé à l'adresse configurée dans ADMIN_APPROVAL_EMAIL. Pour les rôles user et guest, le compte est créé avec approval_status = 'approved' et peut se connecter immédiatement.

9.2 Connexion (POST /api/auth/login)

L'utilisateur s'authentifie via un identifiant unique (e-mail ou nom d'utilisateur) et son mot de passe. Le contrôleur vérifie successivement : l'existence du compte, son statut d'approbation (rejet immédiat avec code 403 si pending ou rejected), son statut actif, puis la correspondance du mot de passe fourni avec le hachage

stocké (bcrypt.compare). En cas de succès, la date de dernière connexion est mise à jour, les jetons d'accès et de rafraîchissement sont émis et déposés en cookies httpOnly, et les informations publiques de l'utilisateur (hors hachage du mot de passe) sont renvoyées au client.

9.3 Déconnexion (POST /api/auth/logout)

La déconnexion supprime simplement les cookies access_token et refresh_token côté client, sans nécessiter d'action côté base de données, conformément au modèle stateless retenu pour les jetons d'accès.

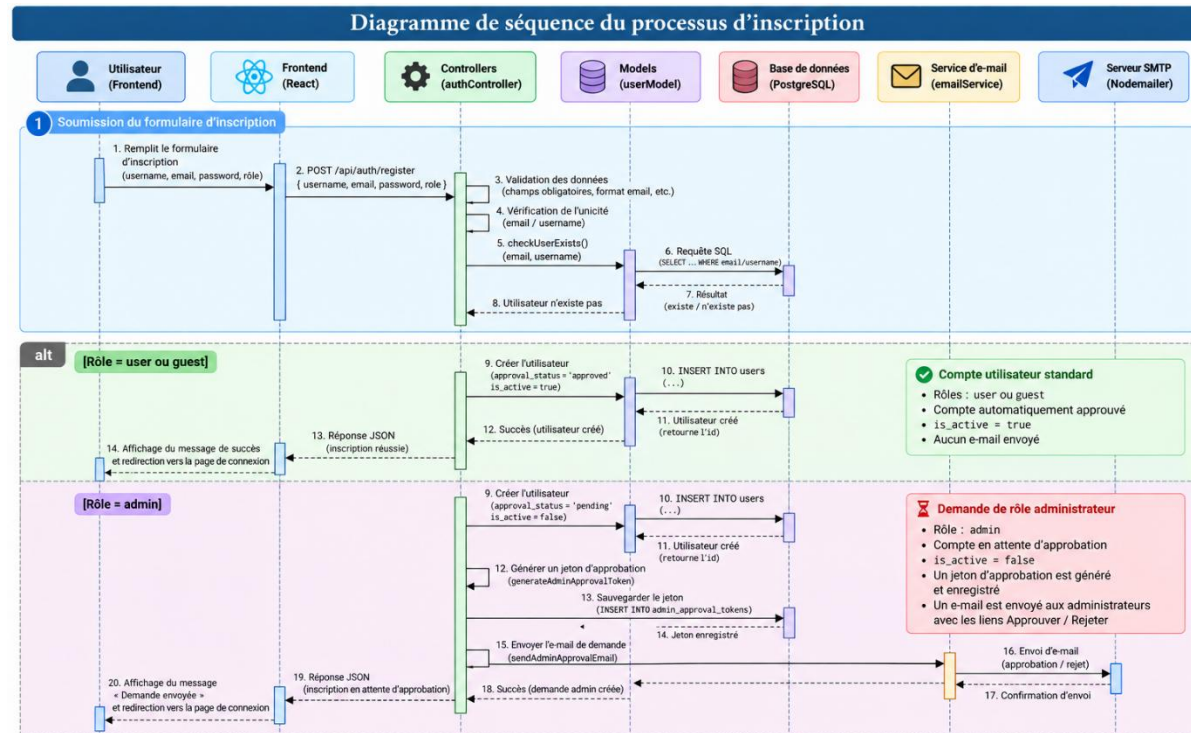


Figure X.7 — Insérer ici le diagramme de séquence du processus d'inscription

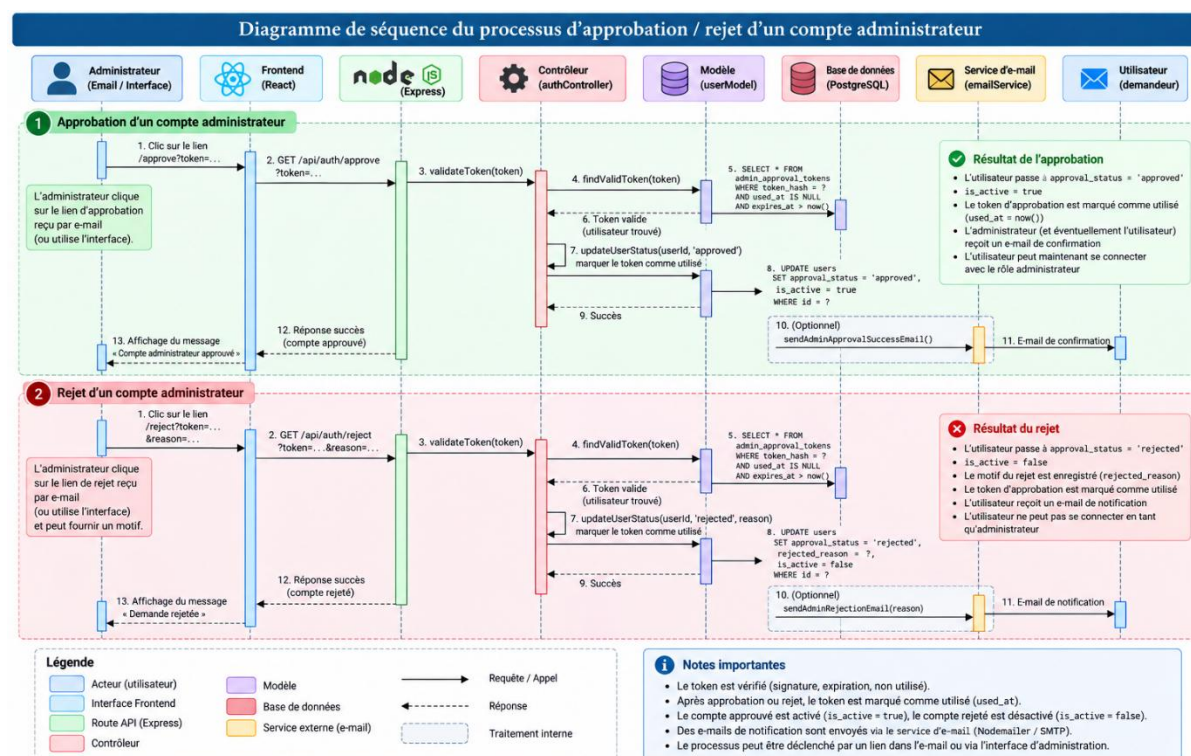


Figure X.8 — Diagramme de séquence du processus d'approbation / rejet d'un compte administrateur

Figure II.8 — diagramme de séquence du processus d'approbation

II.10 Gestion des comptes administrateurs

Afin d'éviter qu'un rôle administrateur puisse être obtenu librement par auto-inscription, le système impose une procédure de validation humaine pour toute demande de compte admin. Ce comportement est gouverné par la variable `ENFORCE_SAFE_PUBLIC_ROLES` : lorsqu'elle vaut `true` (valeur par défaut), toute demande de rôle admin passe par le statut `pending` ; lorsqu'elle vaut `false`, la demande est silencieusement ramenée au rôle par défaut configuré (`DEFAULT_PUBLIC_ROLE`), ce mode étant réservé aux environnements de démonstration ou de test locaux.

10.1 Depuis le tableau de bord administrateur

Un administrateur déjà approuvé peut consulter la liste des demandes en attente (`GET /api/users/admin-requests/pending`) et les approuver (`POST /api/users/admin-requests/:id/approve`) ou les rejeter, avec un motif facultatif (`POST /api/users/admin-requests/:id/reject`). L'approbation réactive le compte (`is_active = true`), enregistre l'identifiant de l'administrateur approbateur et l'horodatage, puis déclenche l'envoi d'un e-mail de confirmation au demandeur. Le rejet maintient le compte désactivé et enregistre le motif fourni.

10.2 Depuis un e-mail : approbation/rejet en un clic

Parallèlement, l'e-mail de notification envoyé à l'adresse `ADMIN_APPROVAL_EMAIL` contient deux liens d'action (« Approuver » / « Rejeter ») pointant vers des points d'entrée non authentifiés (`GET /api/admin-registration/approve` et `/reject`), volontairement accessibles sans connexion préalable puisqu'ils sont destinés à être ouverts directement depuis un client de messagerie. La sécurité de ce mécanisme repose non pas sur une session utilisateur, mais sur la possession d'un jeton aléatoire de 32 octets, à usage unique et expirant après 24 heures, dont seul le hachage SHA-256 est conservé côté serveur. Le traitement de la demande (mise à jour du statut et marquage du jeton comme utilisé) est effectué dans une transaction unique avec verrouillage de ligne (`SELECT ... FOR UPDATE`), ce qui empêche qu'un double clic ou une pré-lecture automatique du lien par un scanner de sécurité côté client de messagerie ne traite la demande deux fois.

Après action, l'administrateur est redirigé vers une page HTML de résultat générée dynamiquement par le backend (succès, rejet ou lien invalide/expiré), et non vers l'interface React, puisque le clic provient directement du client de messagerie.

Diagramme de séquence du flux d'approbation d'un compte administrateur par e-mail

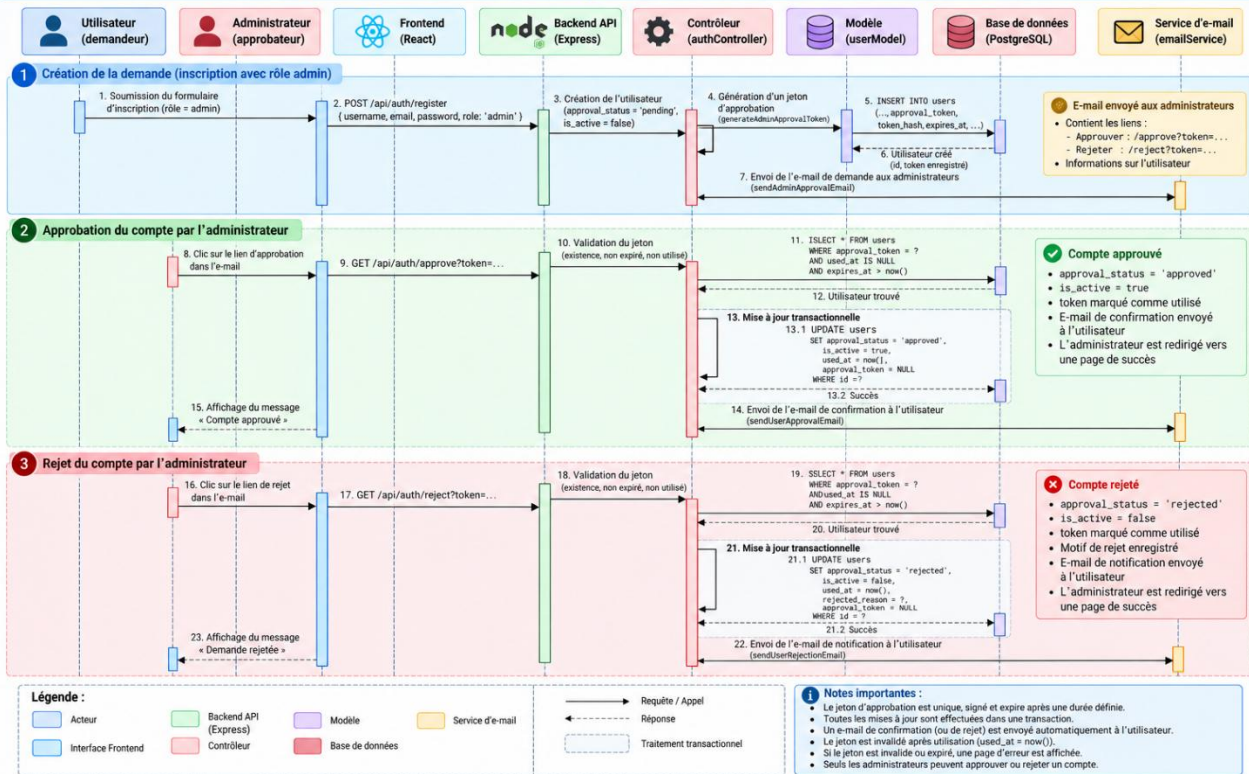


Figure II.9 — diagramme de séquence du flux d'approbation d'un compte administrateur par e-mail

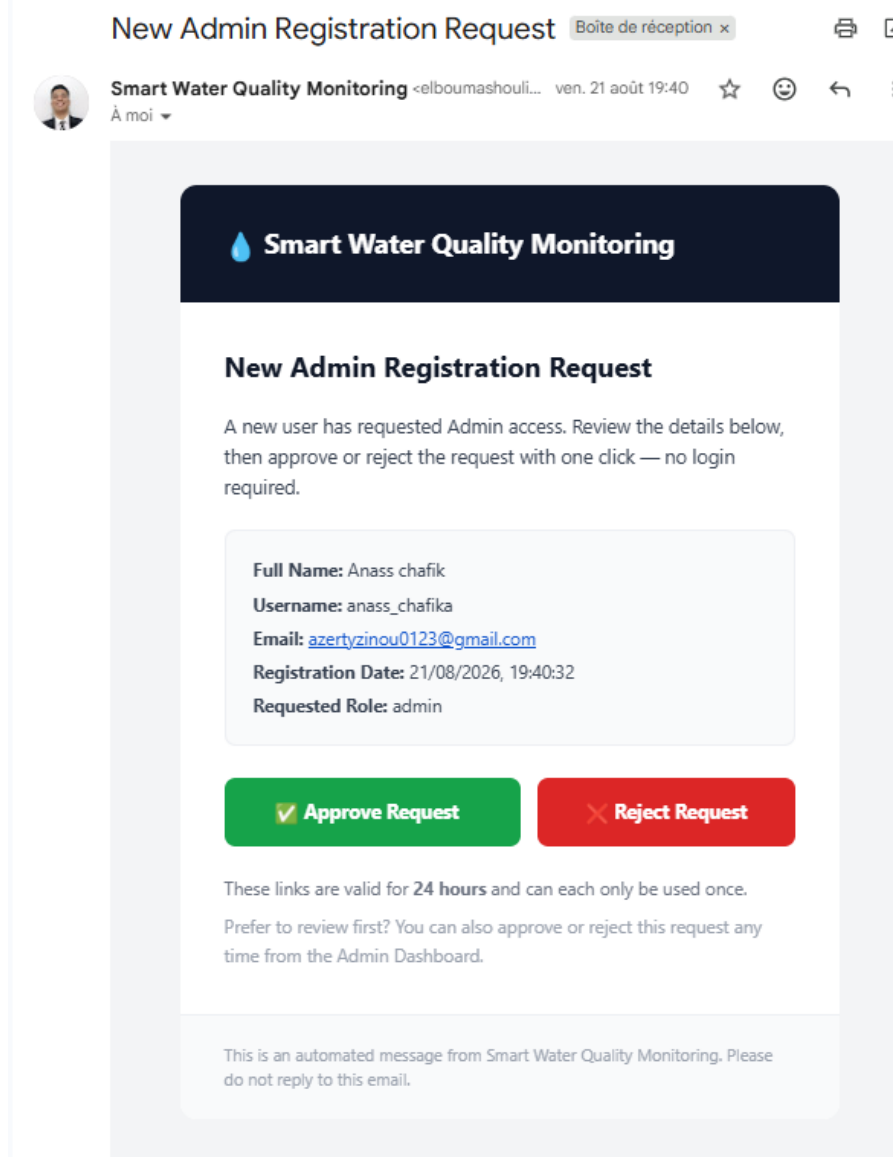


Figure II.10 —capture d'écran de l'e-mail de demande d'approbation d'un compte administrateur, avec les boutons « Approve Request » et « Reject Request ».

II.11 Gestion de la récupération du mot de passe

Le flux « mot de passe oublié » se déroule en deux étapes. Premièrement, `POST /api/auth/forgot-password` reçoit une adresse e-mail : si un compte correspondant existe, un jeton aléatoire est généré, son hachage SHA-256 et sa date d'expiration (15 minutes) sont enregistrés sur la ligne de l'utilisateur (en écrasant tout jeton précédemment émis, ce qui l'invalidé automatiquement), puis un e-mail contenant un lien de réinitialisation (`${FRONTEND_URL}/reset-password/:token`) est envoyé. Dans tous les cas — compte existant, inexistant, ou erreur interne — la réponse renvoyée au client reste rigoureusement identique, afin de ne pas révéler l'existence d'un compte.

Deuxièmement, `POST /api/auth/reset-password` reçoit le jeton brut (extrait de l'URL côté frontend) et le nouveau mot de passe. Le serveur recalcule le hachage SHA-256 du jeton reçu et recherche l'utilisateur correspondant ; il vérifie ensuite que le jeton n'a pas expiré (avec purge du jeton expiré le cas échéant), hache le nouveau mot de passe, puis met à jour le mot de passe et invalide le jeton dans une même transaction atomique (`resetPasswordTransaction`), garantissant qu'un jeton ne peut jamais être réutilisé même en cas de requêtes concurrentes.

Diagramme de séquence du flux complet de récupération de mot de passe

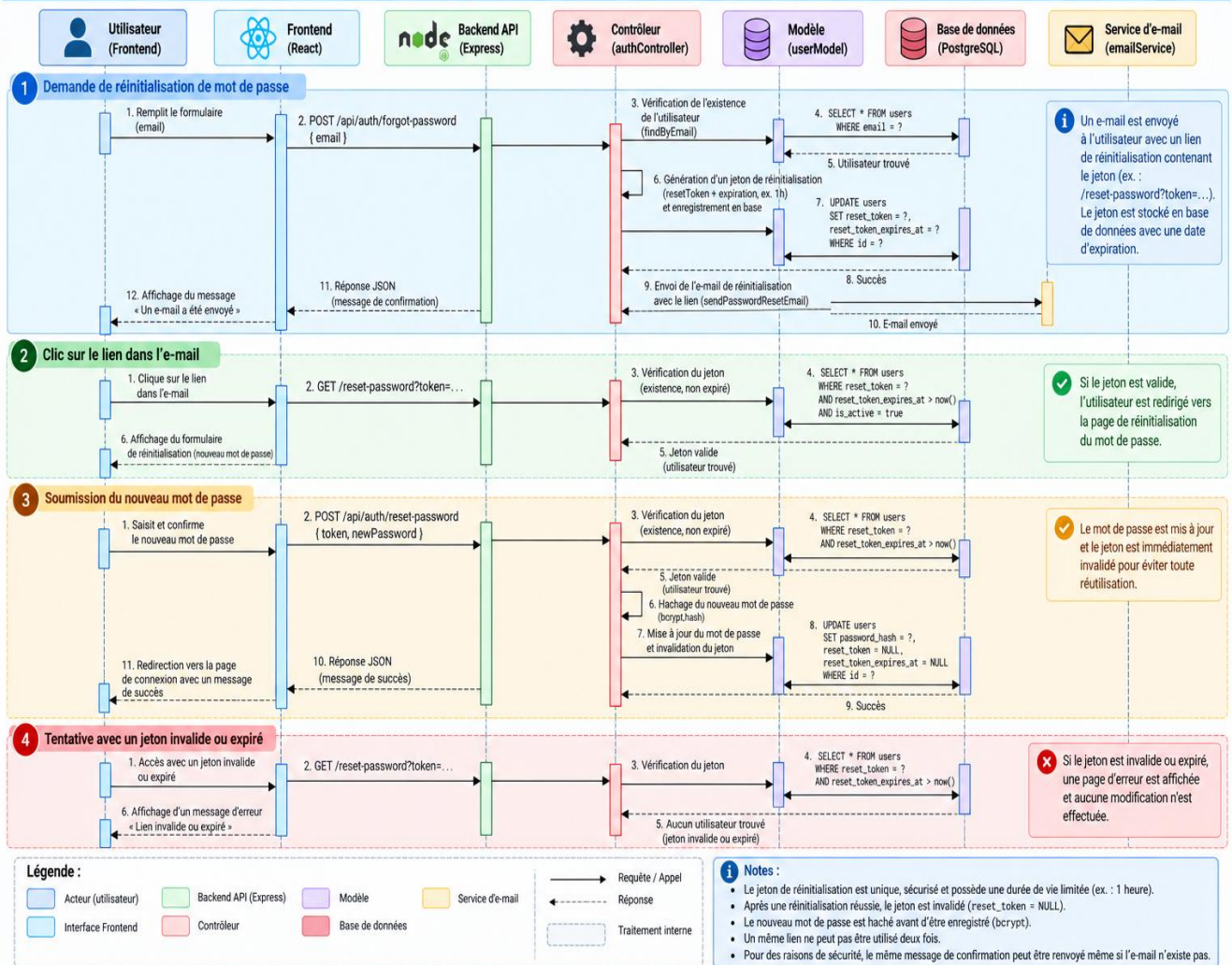


Figure II.11 — diagramme de séquence du flux complet de récupération de mot de passe, de la demande initiale à la réinitialisation effective, incluant la génération et l'invalidation du jeton.

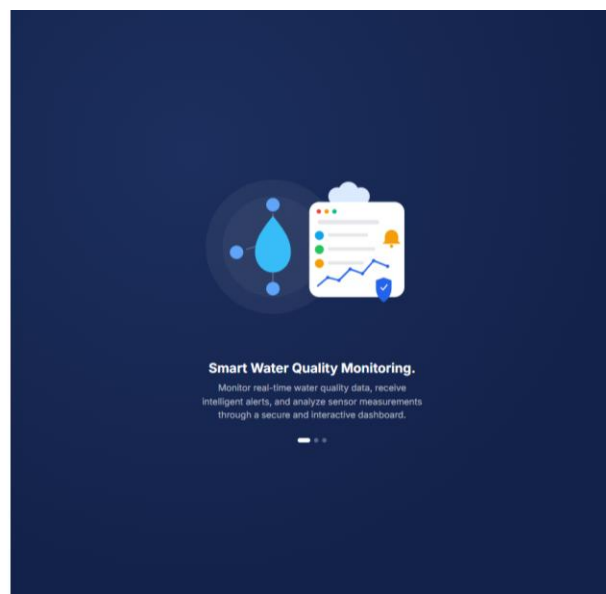
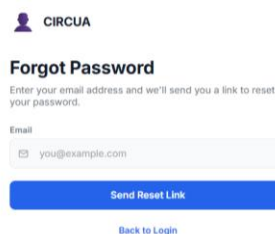


Figure II.12 — capture d'écran des pages « Mot de passe oublié » et « Réinitialisation du mot de passe » côté frontend.

II.12 Communication par email

L'envoi des e-mails transactionnels est assuré par la bibliothèque Nodemailer, configurée via un transporteur SMTP unique et partagé, entièrement paramétré par variables d'environnement (SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD, SMTP_FROM). Le système prend également en charge, de façon optionnelle, le routage du trafic SMTP sortant à travers un proxy HTTP ou SOCKS5 (variable SMTP_PROXY), utile dans les environnements réseau où les connexions SMTP directes sont bloquées ; ce point est documenté dans le fichier PROXY_SETUP.md fourni avec le projet, et un script autonome de diagnostic (smtp_diagnose.js) permet de tester successivement la connectivité TCP brute, l'authentification SMTP et l'envoi effectif d'un e-mail de test.

12.1 Types d'e-mails envoyés

Chaque e-mail est généré à la fois en version HTML (gabarit à mise en page responsive, en tableaux HTML pour la compatibilité maximale entre clients de messagerie) et en version texte brut, sous une identité visuelle cohérente reprenant le nom et les couleurs du projet « Smart Water Quality Monitoring ». L'échec d'envoi d'un e-mail non critique (confirmation d'approbation/rejet, par exemple) est journalisé côté serveur mais ne fait jamais échouer l'opération métier associée, qui a déjà été appliquée en base de données.

II.13 API et flux de communication

L'ensemble des points d'entrée de l'API est préfixé par /api. Le tableau suivant récapitule les routes exposées par le module.

Le point d'entrée GET /api/health, hors périmètre des trois routeurs ci-dessus, permet de vérifier rapidement la disponibilité du service (statut et horodatage), utile pour la supervision ou les vérifications de déploiement.

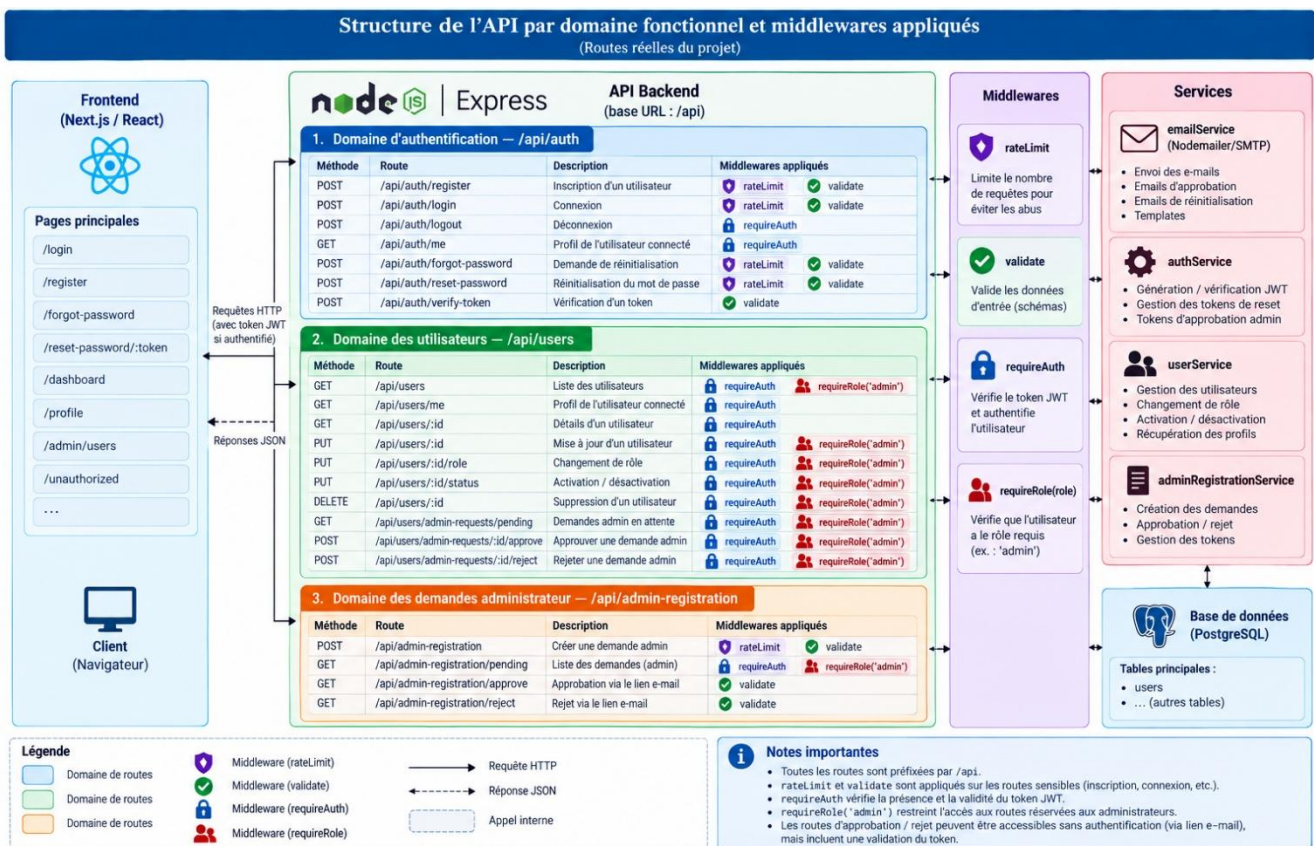


Figure II.13 — diagramme (par exemple un diagramme de composants ou un schéma de flux) représentant la structure de l'API par domaine fonctionnel

II .14 Tests et validation du système

Le projet, dans son état actuel, ne comporte pas de suite de tests automatisés (aucun framework de test — Jest, Mocha, Supertest ou équivalent — n'est déclaré dans les dépendances du backend ou du frontend, et aucun répertoire ou fichier de test n'est présent dans l'arborescence du projet). La validation du module a donc été réalisée de manière manuelle et fonctionnelle, par exécution des scénarios ci-dessous en environnement de développement local.

14.1 Scénarios de validation fonctionnelle réalisés

- Inscription avec des données valides (rôle user) suivie d'une connexion immédiate réussie
- Inscription avec un e-mail ou un nom d'utilisateur déjà utilisé : vérification du rejet avec code HTTP 409
- Inscription avec un mot de passe ne respectant pas la politique de complexité : vérification des messages de validation renvoyés par express-validator
- Inscription avec le rôle admin : vérification de la mise en attente du compte (approval_status = pending, is_active = false) et de l'envoi de l'e-mail de notification
- Tentative de connexion sur un compte en attente d'approbation ou rejeté : vérification du code HTTP 403 et du message associé
- Connexion avec des identifiants erronés : vérification du message générique « Invalid credentials »
- Déconnexion : vérification de la suppression des cookies de session et de la perte d'accès aux routes protégées
- Demande de réinitialisation de mot de passe pour un e-mail existant et pour un e-mail inexistant : vérification que la réponse renvoyée est identique dans les deux cas
- Réinitialisation du mot de passe avec un jeton valide, puis nouvelle tentative avec le même jeton : vérification que la seconde tentative échoue (jeton à usage unique)
- Réinitialisation avec un jeton expiré (au-delà de 15 minutes) : vérification du rejet
- Approbation d'une demande de compte admin depuis le tableau de bord : vérification de l'activation du compte et de la réception de l'e-mail de confirmation
- Approbation et rejet via les liens envoyés par e-mail : vérification du traitement correct et de l'affichage de la page de résultat HTML
- Réutilisation d'un lien d'approbation/rejet déjà utilisé ou expiré : vérification de l'affichage de la page « lien invalide ou expiré »
- Tentative d'accès à une route d'administration (/api/users, /admin/users) avec un compte non-admin : vérification du refus (403 côté API, redirection vers /unauthorized côté frontend)
- Tentative de modification du rôle ou du statut de son propre compte par un utilisateur non-administrateur : vérification du refus
- Déclenchement répété du rate limiter sur les routes de connexion et de mot de passe oublié : vérification du blocage temporaire au-delà du seuil configuré

II.15 Déploiement et configuration

Le module est conçu pour être déployé indépendamment du reste de l'application, à condition de disposer d'une instance PostgreSQL 13+ et d'un compte SMTP valide. Le backend nécessite Node.js 18 ou supérieur.

15.1 Mise en place du backend

```
cd backend
npm install
cp .env.example .env      # puis renseigner les valeurs réelles
createdb auth_app_db
npm run migrate           # applique les migrations SQL versionnées
npm run dev               # ou : npm start
```

15.2 Mise en place du frontend

```
cd frontend
npm install
cp .env.example .env
npm run dev
```

II.18 Conclusion

Ce chapitre a présenté la conception et la réalisation d'un module complet d'authentification et de gestion des utilisateurs, structuré autour d'une architecture claire (React côté client, Express/PostgreSQL côté serveur) et de mécanismes de sécurité éprouvés : hachage bcrypt des mots de passe, authentification par jetons JWT transmis en cookies httpOnly, contrôle d'accès basé sur les rôles systématiquement revérifié côté serveur, jetons à usage unique hachés en base pour les opérations sensibles, et limitation de débit sur l'ensemble des routes exposées au public.

Au-delà des fonctionnalités classiques d'inscription, de connexion et de récupération de mot de passe, le module se distingue par la mise en place d'un flux original d'approbation des comptes administrateurs, combinant validation depuis un tableau de bord et action en un clic depuis un e-mail sécurisé par jeton. Cette réalisation a permis de mettre en pratique, à l'échelle d'un module réel destiné à protéger l'accès à la plateforme de supervision de la qualité de l'eau, l'ensemble des bonnes pratiques de sécurité applicative étudiées au cours de la formation, tout en identifiant clairement les axes d'amélioration restant à explorer, en particulier l'automatisation des tests et le renforcement de la traçabilité des actions d'administration.

Conclusion générale :

Le projet Smart Water Quality System est un système à deux niveaux complet et fonctionnel : un backend FastAPI qui ingère de façon fiable la télémétrie des capteurs (y compris un rattrapage complet de l'historique ThingSpeak au démarrage), modélise le domaine étude/cycle/alerte dans PostgreSQL — avec, depuis la dernière migration, une relation many-to-many entre études et cycles permettant leur réutilisation sans duplication — et expose ce modèle via une API REST documentée ; et un frontend Next.js qui offre aux opérateurs de laboratoire une interface focalisée, proche d'un tableur, pour saisir et consulter les résultats de cycle, importer des données en masse, réutiliser des cycles existants entre plusieurs études, et surveiller la santé de la collecte.

Les deux moitiés communiquent exclusivement via une API JSON bien définie et protégée par CORS, et la logique de calcul (moyenne, écart-type d'échantillon, pourcentage d'abattement) est délibérément dupliquée de façon contrôlée — une fois de manière authoritative côté backend, une fois pour un retour instantané côté navigateur — de sorte que les deux ne peuvent jamais diverger au-delà d'un arrondi d'affichage.

Les principales limites actuelles sont d'ordre opérationnel plutôt qu'architectural : absence d'authentification, absence de suite de tests automatisés, absence d'outillage de déploiement. Ce sont des étapes suivantes réalistes et bien délimitées, et non les signes d'un modèle métier inachevé.